ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN A1

HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH – CO5085

CNN vs TRANSFORMER TRÊN TẬP ẢNH LỚN

Đánh giá thực nghiệm CNN và Transformer dưới các chiến lược huấn luyện khác nhau

Giảng viên phụ trách: TS. Lê Thành Sách
TS. Nguyễn Quốc Minh
Lớp – Nhóm: CH01 – DL4DL
Sinh viên thực hiện: Lê Tiến Đạt – 2310653
Nguyễn Lê Thảo Ly – 2312010

Thành phố Hồ Chí Minh, Tháng 11/2026

MỤC LỤC

1 GIỚI THIỆU **7**
1.1 Bối cảnh . . . 7
1.2 Mục tiêu và câu hỏi nghiên cứu . . . 7
1.3 Phạm vi . . . 8
1.4 Nguyên tắc so sánh . . . 8
1.5 Liên kết dự án . . . 8
1.6 Cấu trúc báo cáo . . . 9

2 PHÂN TÍCH TẬP DỮ LIỆU **10**
2.1 Food-101 . . . 10
2.2 Giấy phép và phạm vi sử dụng . . . 10
2.3 Chia dữ liệu . . . 11
2.4 Phân tích khám phá (EDA) . . . 12
2.5 Chất lượng nhãn . . . 14
2.6 Tiền xử lý, tăng cường dữ liệu và bộ nạp dữ liệu . . . 15

3 CÁC MÔ HÌNH CNN VÀ TRANSFORMER **16**
3.1 Kiến trúc CNN . . . 17
3.2 Kiến trúc Transformer . . . 17
3.3 Lý do chọn và inductive bias . . . 17
3.4 Các chế độ đóng băng . . . 18
3.5 Lưu ý khi diễn giải . . . 18

4 CHIẾN LƯỢC HUẤN LUYỆN **19**
4.1 Huấn luyện từ đầu (from-scratch) . . . 19
4.2 Sử dụng mô hình pretrained . . . 19
4.2.1 Đóng băng toàn bộ backbone (linear probing) . . . 19
4.2.2 Fine-tune một phần . . . 20
4.2.3 Fine-tune toàn bộ . . . 20

4.3 So sánh các chiến lược . . . 20
4.3.1 Tham số được cập nhật . . . 20
4.3.2 Chi phí tính toán (đo kiểm soát) . . . 20
4.3.3 Ưu điểm và hạn chế . . . 22

5 THIẾT KẾ THỰC NGHIỆM 23
5.1 Biến nghiên cứu . . . 23
5.2 Ma trận thí nghiệm . . . 23
5.3 Công thức huấn luyện chi tiết . . . 24
5.4 Quét learning rate . . . 26
5.5 Các thí nghiệm và ngân sách tính toán . . . 26
5.6 Thí nghiệm bổ sung sau giao thức . . . 27
5.7 Giao thức đánh giá và phân tích . . . 28
5.8 Cách đo chi phí . . . 29

6 KẾT QUẢ THỰC NGHIỆM 30
6.1 Độ chính xác và F1 . . . 31
6.1.1 Kết quả của hai kiến trúc mở rộng . . . 32
6.2 Diễn biến quá trình huấn luyện . . . 33
6.3 Phân tích lỗi . . . 34
6.4 Chi phí tính toán . . . 36
6.4.1 Chi phí huấn luyện . . . 37
6.4.2 Chi phí suy luận . . . 38
6.5 Ablation theo kích thước dữ liệu . . . 38
6.6 Kết quả phân tích bổ sung (ngoài yêu cầu tối thiểu) . . . 39
6.6.1 Hiệu chỉnh độ tin cậy . . . 40
6.6.2 Ensemble . . . 41
6.6.3 Độ bền vững trước nhiễu . . . 41
6.6.4 Diễn giải: Grad-CAM và ảnh huấn luyện loss cao . . . 44
6.7 Thí nghiệm bổ sung sau giao thức . . . 45
6.7.1 LR của freeze một phần và LR ở biên . . . 45
6.7.2 Nguồn gốc của sự thiếu tự tin . . . 48
6.7.3 Layer-wise LR decay . . . 48

7 PHÂN TÍCH VÀ THẢO LUẬN 49
7.1 CNN và Transformer . . . 49
7.2 Vai trò của pretraining . . . 50
7.3 Ảnh hưởng của fine-tuning và freeze . . . 50

7.4 Phân tích lỗi . . . 51
7.4.1 Hiệu năng theo lớp . . . 51
7.4.2 Các mẫu dự đoán sai . . . 51
7.4.3 Kiểu lỗi của CNN và Transformer . . . 51
7.5 Hiệu chỉnh, ensemble, độ bền vững và diễn giải . . . 52
7.6 Đánh đổi giữa hiệu năng và chi phí . . . 52

8 TÁI LẬP THÍ NGHIỆM 53

9 KẾT LUẬN 56
9.1 Tổng kết kết quả . . . 56
9.2 Hạn chế và hướng phát triển . . . 57

TÀI LIỆU THAM KHẢO 59

A Ví dụ tệp cấu hình một lần chạy 61

B Kết quả thực nghiệm bổ sung 63
B.1 Quét learning rate . . . 63
B.1.1 Điểm quét bổ sung sau giao thức . . . 64
B.2 Kết quả từng seed . . . 66
B.3 Chi phí và hiệu năng của chế độ freeze . . . 66

C Khai báo sử dụng công cụ AI 67
C.1 Công cụ và phạm vi . . . 67
C.2 Những gì AI không làm . . . 67
C.3 Cách nhóm kiểm tra . . . 68
C.4 Hạn chế . . . 68

DANH SÁCH BẢNG

2.1 Chia dữ liệu Food-101 của nhóm. 11

3.1 Các mô hình được so sánh (tham số tính với đầu ra 101 lớp). 16
3.2 Phần được huấn luyện ở mỗi chế độ và tỷ lệ tham số trainable (đo trên mô hình). 18

4.1 Chi phí tương đối của các chế độ so với full fine-tuning (Tesla T4, AMP fp16, batch 64, batch tổng hợp trên GPU; đo bằng `cost_bench.py`). 21
4.2 Tóm tắt kỳ vọng về các chiến lược (sẽ được kiểm chứng ở Chương 7). 22

5.1 Biến độc lập, biến phụ thuộc và các yếu tố được giữ cố định. 23
5.2 Tám cấu hình lõi (cặp 1). 24
5.3 Công thức huấn luyện chính xác. 24
5.4 Lưới learning rate đỉnh cho mỗi mô hình. 26
5.5 Thí nghiệm theo thứ tự ưu tiên và chi phí ước tính. 27

6.1 Độ chính xác trên tập test (25 250 ảnh); mean ± std trên 3 seed với cấu hình pretrained, 1 seed với from-scratch. 31
6.2 Kết quả của ConvNeXt-T và Swin-T trên tập test (1 seed; pretrained 10 epoch, from-scratch 40 epoch). 32
6.3 Các lần chạy mở rộng và đối chứng 100 epoch (1 seed). 33
6.4 Các cặp lớp bị nhầm nhiều nhất trên test (full fine-tuning, seed 0). Cột "Chung" là các cặp ResNet-50 mà DeiT-S cũng nhầm, đếm theo ResNet-50. 35
6.5 Các lớp có độ chính xác thấp nhất (%, test, seed 0). 35
6.6 Chi phí huấn luyện. Cột "chỉ GPU" dùng để so sánh (batch tổng hợp trên GPU); cột "thực tế" là thời gian mỗi epoch trong lần huấn luyện thật trên Kaggle (hai tiến trình chạy song song, mỗi tiến trình một GPU), bị giới hạn bởi bộ nạp dữ liệu (xem Mục 5.8). Peak mem là bộ nhớ đỉnh khi huấn luyện trong phép đo kiểm soát (batch 64). 37

6.7 Chi phí suy luận (chỉ phụ thuộc kiến trúc; AMP fp16; Tesla T4, PyTorch 2.11.0+cu128, timm 1.0.30, đo bằng `cost_bench.py`). Latency batch 1 bị chi phối bởi chi phí khởi chạy kernel nên không tỷ lệ với GMACs. 38
6.8 Top-1 test (%) theo tỷ lệ dữ liệu huấn luyện; cột cuối là mức tăng từ 10% lên 100% dữ liệu (điểm phần trăm). Một seed. 38
6.9 Hiệu chỉnh trên test, trước và sau temperature scaling (nhiệt độ T khớp trên logits validation). ECE tính theo phần trăm, 15 bin. 40
6.10 Top-1 test (%) của ensemble trung bình xác suất. 41
6.11 Độ chính xác còn lại (%) ở mức nhiễu 3 so với ảnh sạch. Cột "Sạch" là top-1 (%) trên tập con. 42
6.12 Độ chính xác còn lại (%) ở mức nhiễu 3, mean ± std trên 3 seed. 42
6.13 Các lần chạy bổ sung 10 epoch so với lần chạy giao thức cùng mô hình (seed 0, trừ khi ghi rõ). ECE tính theo phần trăm. 47

8.1 Quy trình chuẩn và cách nhóm thực sự chạy. 54

B.1 Kết quả quét learning rate. 63
B.2 Điểm quét LR bổ sung: top-1 validation (%). 65
B.3 Top-1 test (%) của từng seed (cấu hình pretrained, 10 epoch) và độ lệch chuẩn. . 66
B.4 Độ chính xác so với full fine-tuning và chi phí của từng chế độ. 66

DANH SÁCH HÌNH VẼ

2.1 Số ảnh mỗi lớp của ba tập. Mọi lớp có cùng số ảnh trong từng tập (cực tiểu bằng cực đại), nên không cần reweighting. 12
2.2 Mẫu ảnh huấn luyện Food-101: sáu lớp được chọn theo quy tắc cố định (lớp thứ $0, 17, 34, 51, 68, 85$), sáu ảnh mỗi lớp. 13
2.3 Ảnh gốc (cột đầu) và năm lần augmentation (RandomResizedCrop, lật ngang, RandAugment) của cùng một ảnh, dùng chính seed theo mẫu của bộ huấn luyện. 14

6.1 Độ chính xác validation theo epoch của các chế độ huấn luyện (lần chạy seed 0). . 33
6.2 Độ chính xác test theo thời gian huấn luyện mỗi epoch chỉ GPU. 34
6.3 Độ chính xác theo lớp: ResNet-50 so với DeiT-S, và các lớp khó nhất. 36
6.4 Độ chính xác theo tỷ lệ dữ liệu huấn luyện. 39
6.5 Biểu đồ reliability trước và sau temperature scaling. 40
6.6 Độ chính xác theo mức nhiễu của từng loại. 43
6.7 Grad-CAM của ResNet-50 và DeiT-S (full fine-tuning, seed 0) trên các ảnh test được chọn trước (một ảnh đầu tiên của mỗi lớp trong tập con xác định); đây là diễn giải định tính. 44
6.8 Ảnh huấn luyện có loss cao nhất và nhãn gốc/dự đoán của hai mô hình. 45

B.1 Top-1 validation theo learning rate. 64

Chương 1

GIỚI THIỆU

1.1 Bối cảnh

Mạng tích chập (CNN) và Vision Transformer (ViT) là hai họ kiến trúc chủ đạo trong thị giác máy tính. CNN mang sẵn *inductive bias* về tính cục bộ và bất biến tịnh tiến; ViT dựa trên chú ý toàn cục và thường cần nhiều dữ liệu hoặc trọng số pretrained để đạt hiệu năng cao [3, 14]. Khi dữ liệu và tài nguyên tính toán hạn chế, câu hỏi thực tế là nên chọn họ kiến trúc nào, nên khởi tạo từ đâu, và nên cập nhật bao nhiêu phần của mô hình.

1.2 Mục tiêu và câu hỏi nghiên cứu

Bài tập lớn A1 so sánh CNN và Transformer trên một tập ảnh lớn do nhóm chọn, dưới nhiều chế độ huấn luyện: huấn luyện từ đầu (*from-scratch*) so với dùng trọng số pretrained, và ba kiểu đóng băng khi dùng pretrained (đóng băng toàn bộ backbone, đóng băng một phần, fine-tune toàn bộ). Nhóm tìm câu trả lời cho bốn câu hỏi:

RQ1. *Khi nào chiến lược freeze có lợi?* Đánh đổi giữa chi phí tính toán, nguy cơ overfitting và độ chính xác.

RQ2. *Khi nào ViT cần pretrain?* Khoảng cách giữa from-scratch và pretrained thay đổi thế nào theo kích thước dữ liệu huấn luyện, và so với CNN ra sao.

RQ3. *Chi phí tài nguyên:* số tham số trainable, GMACs, thời gian huấn luyện, thông lượng và độ trễ suy luận, bộ nhớ GPU đỉnh.

RQ4. *Phân tích lỗi và độ tin cậy:* nhầm lẫn giữa các lớp, khác biệt lỗi giữa CNN và ViT, hiệu chỉnh độ tin cậy (calibration), tính bổ trợ khi ensemble và độ bền vững trước nhiễu.

1.3 Phạm vi

Nhóm chọn tập **Food-101** [1] (101 lớp món ăn, khoảng 100 nghìn ảnh) và so sánh ResNet-50 [7, 16] với DeiT-Small [14] (cặp 1, phần cam kết). Cặp ConvNeXt-Tiny [10] và Swin-Tiny [9] (cặp 2) là phần mở rộng ngoài phần cam kết; nhóm đã chạy được cả hai (một seed). Các thí nghiệm gồm tám cấu hình lõi, ablation theo kích thước dữ liệu, và lặp lại ba seed cho các cấu hình pretrained. Sau đó là các phân tích chỉ cần suy luận (hiệu chỉnh, ensemble, độ bền vững, diễn giải) được trình bày như phần mở rộng ngoài yêu cầu tối thiểu của đề bài. Báo cáo cũng trình bày quy trình tái lập, tách bạch giữa *quy trình chuẩn* (chỉ cần mã nguồn và báo cáo) và *cách nhóm thực sự chạy* trên Kaggle (hai GPU Tesla T4 mỗi phiên, kết quả và checkpoint lưu trên một kho Hugging Face), cùng một notebook Colab để tái hiện kết quả.

1.4 Nguyên tắc so sánh

Để chênh lệch kết quả quy được cho kiến trúc và chế độ huấn luyện, nhóm giữ cố định các yếu tố sau cho mọi mô hình:

- cùng split dữ liệu (cố định bằng `split.json`) và cùng phép biến đổi khi đánh giá;
- cùng một công thức huấn luyện (tối ưu hóa, lịch learning rate, augmentation, regularization), chỉ khác learning rate đỉnh được chọn bằng cùng một quy trình quét trên validation;
- cùng số epoch theo chế độ (10 epoch cho pretrained, 40 epoch cho from-scratch) và báo cáo epoch cuối, không chọn checkpoint tốt nhất;
- hai mô hình trong mỗi cặp có số tham số và GMACs xấp xỉ nhau (Bảng 3.1);
- cùng một công cụ, quy ước và GPU để đo chi phí (Mục 5.8).

Những điểm không thể đồng nhất, như công thức pretrain khác nhau của từng bộ trọng số, được nêu ở Mục 9.2.

1.5 Liên kết dự án

Các tài nguyên liên quan đến bài tập A1:

- Kho mã nguồn GitHub (thư mục `A1`): `https://github.com/broistg/deep-learning-CO5085/tree/main/A1`
- GitHub Page của A1: `https://broistg.github.io/deep-learning-CO5085/assignments/a1.html`

- Landing page của nhóm: `https://broistg.github.io/deep-learning-CO5085/index.html`
- Trọng số và logits (GitHub Release `weights-v1`): `https://github.com/broistg/deep-learning-CO5085/releases/tag/weights-v1`

1.6 Cấu trúc báo cáo

Chương 2 mô tả tập dữ liệu, tiền xử lý và bộ nạp dữ liệu; Chương 3 giới thiệu kiến trúc các mô hình; Chương 4 trình bày các chiến lược huấn luyện; Chương 5 nêu thiết kế thực nghiệm; Chương 6 trình bày kết quả; Chương 7 phân tích và thảo luận; Chương 8 trình bày cách tái lập; Chương 9 kết luận và nêu hạn chế.

Trạng thái tài liệu. Đây là bản báo cáo A1.2 hoàn chỉnh. Các Chương 2–5 và Chương 8 giữ nội dung của mốc A1.1 (dataset và protocol, đã cập nhật theo kết quả thực chạy); Chương 6, 7 và 9 là phần mới của mốc A1.2 (kết quả, thảo luận và kết luận).

Chương 2

PHÂN TÍCH TẬP DỮ LIỆU

2.1 Food-101

Food-101 [1] gồm 101 lớp món ăn, mỗi lớp 1 000 ảnh: 750 ảnh huấn luyện và 250 ảnh kiểm thử, tổng cộng 101 000 ảnh. Ảnh được thu thập từ Foodspotting; theo tác giả, ảnh huấn luyện cố ý không được làm sạch nên còn chứa nhiễu (kể cả nhiễu nhãn), còn ảnh kiểm thử được rà soát nhãn thủ công. Trang dữ liệu: `https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/`; bản trên Hugging Face: `https://huggingface.co/datasets/ethz/food101`.

Lý do chọn.

- 101 lớp phân loại mịn với khoảng 75 nghìn ảnh huấn luyện: lớn và khó hơn các tập đã dùng ở bài E1, đủ để khác biệt giữa CNN và ViT, và tác dụng của pretrain/freeze, có ý nghĩa.
- Các lớp cân bằng tuyệt đối nên không cần reweighting, oversampling hay focal loss.
- Cạnh dài tối đa 512 px, phù hợp đầu vào 224×224 và chi phí tải dữ liệu thấp trên các nền tảng miễn phí.
- Ảnh được thu thập từ Foodspotting, không phải từ ImageNet. Tuy nhiên nhóm *chưa kiểm chứng độc lập* mức độ trùng lặp ở cấp ảnh giữa Food-101 và dữ liệu pretrain (ImageNet-1k), nên không khẳng định rằng các mô hình pretrained chưa từng thấy ảnh kiểm thử. ImageNet-1k có một số lớp liên quan đến món ăn, vì vậy lợi thế của pretrain có thể một phần đến từ sự gần gũi về miền dữ liệu (xem Mục 9.2).

2.2 Giấy phép và phạm vi sử dụng

Giấy phép của Food-101 là **chưa rõ**: dataset card trên Hugging Face ghi license là "unknown" [4]. Phần Licensing Information của thẻ này nêu rằng ảnh đến từ Foodspotting, không thuộc sở hữu

của ETH Zürich, và mọi việc sử dụng vượt quá "scientific fair use" phải được thỏa thuận với chủ sở hữu ảnh theo điều khoản của Foodspotting [4]. Theo bản lưu trữ ngày 23/12/2017 của điều khoản Foodspotting (hiệu lực từ 17/10/2012) [5], người đóng góp giữ bản quyền nội dung, Foodspotting được cấp quyền sử dụng rộng (gồm quyền cấp phép lại), còn việc bên thứ ba sử dụng nội dung cần văn bản cho phép của Foodspotting hoặc chủ sở hữu.

Vì nhóm chưa kiểm chứng độc lập phạm vi giấy phép áp dụng cho bản phân phối Food-101 và cho cách sử dụng của nhóm, tập dữ liệu **không** được coi là tài nguyên mở cho việc sử dụng và phân phối không hạn chế. Nhóm chỉ dùng dữ liệu cho mục đích học tập và nghiên cứu trong bài tập lớn của môn học, và **không công bố** ảnh gốc, ảnh đã resize, ảnh đã tăng cường hoặc làm nhiễu, hay bất kỳ cache nào chứa ảnh; các bản này được sinh lại bằng mã nguồn với seed cố định. Nhóm công bố mã nguồn, cấu hình, trọng số mô hình và logits dự đoán (không chứa ảnh). Báo cáo chỉ chèn một số ít ảnh minh họa cỡ nhỏ khi cần trình bày kết quả phân tích. Mọi hình thức phân phối công khai khác chỉ được xem xét sau khi làm rõ điều kiện cấp phép. Nguồn dữ liệu: [1].

2.3 Chia dữ liệu

Tập train chính thức (750 ảnh mỗi lớp) được chia phân tầng thành 700 ảnh huấn luyện và 50 ảnh validation mỗi lớp bằng seed 42; tập test chính thức được giữ nguyên và không dùng để huấn luyện hay chọn siêu tham số. Danh sách ảnh được lưu trong `split.json` (SHA-256, 16 ký tự đầu: `75ed81bfe380c089`) và được commit cùng mã nguồn; chương trình kiểm tra giá trị này khi khởi động.

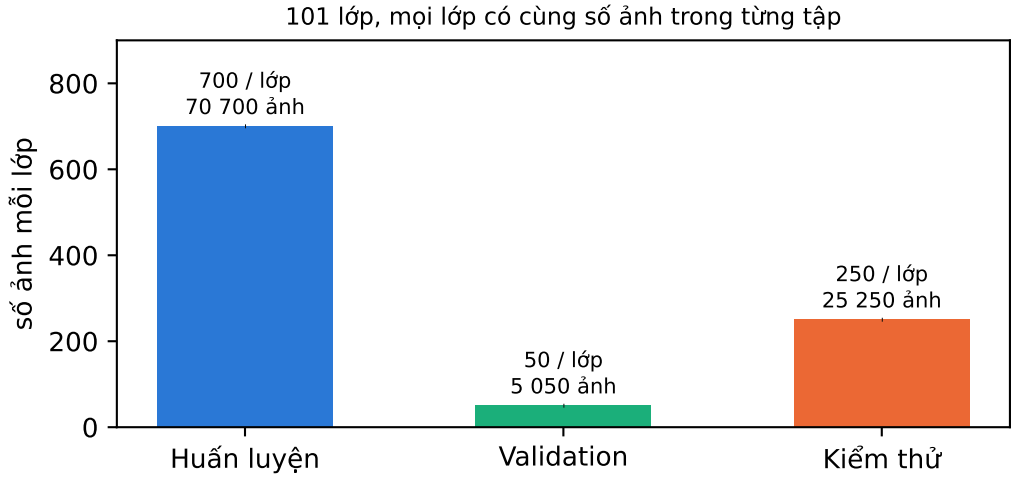
Bảng 2.1: Chia dữ liệu Food-101 của nhóm.

Tập	Số ảnh	Ảnh/lớp	Cách tạo
Huấn luyện	70 700	700	Phần còn lại của tập train chính thức
Validation	5 050	50	Lấy phân tầng từ tập train chính thức, seed 42
Kiểm thử	25 250	250	Tập test chính thức, giữ nguyên

Validation chỉ dùng để chọn learning rate và để fit temperature khi hiệu chỉnh độ tin cậy. Do validation lấy từ phần train có nhiễu nhãn, nó chỉ được dùng cho các mục đích này; mọi số liệu báo cáo cuối cùng là trên tập test, ở epoch cuối của mỗi lần chạy (không chọn checkpoint tốt nhất theo validation).

2.4 Phân tích khám phá (EDA)

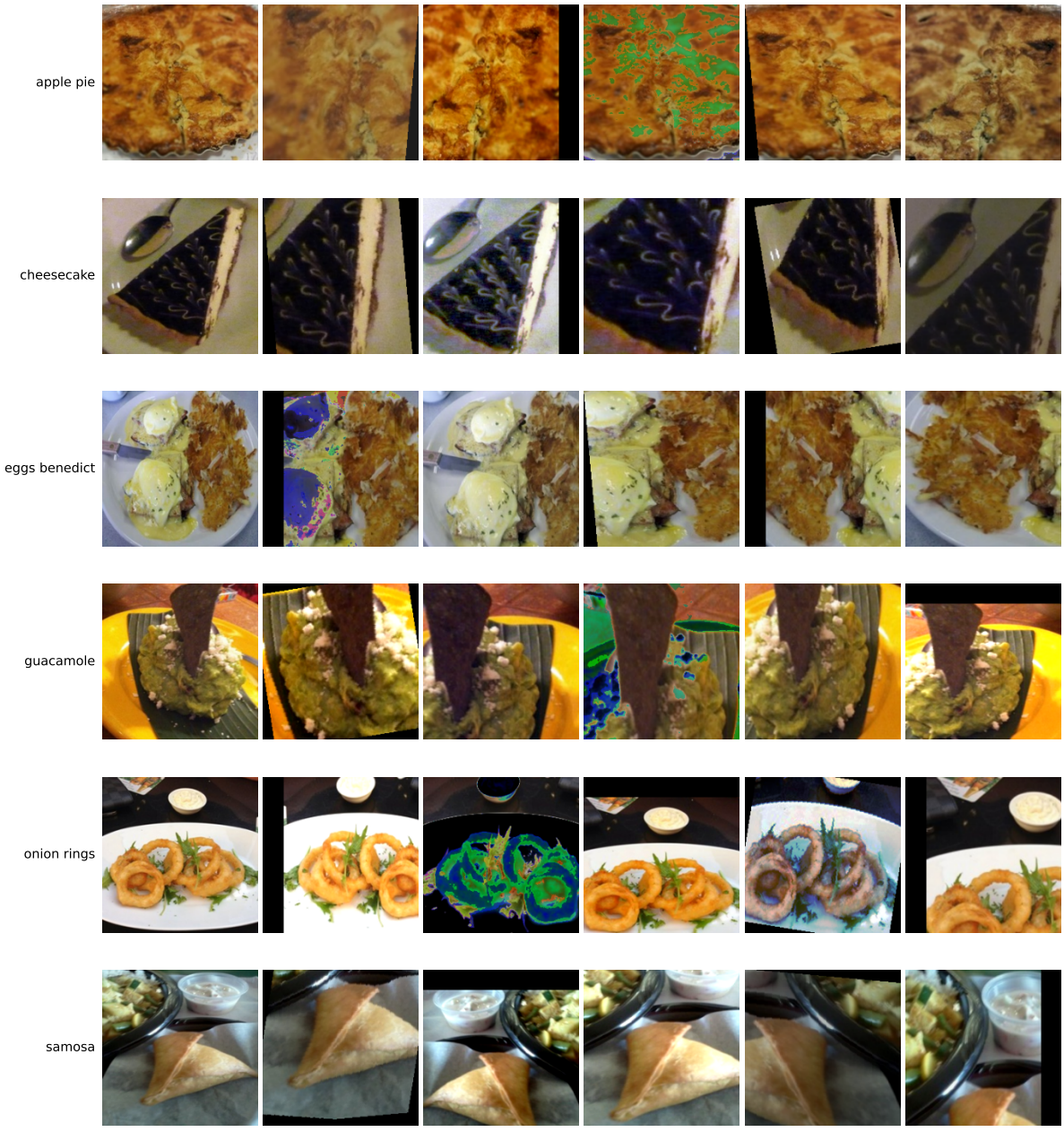
Trên một mẫu ngẫu nhiên 2 000 ảnh của bản Food-101 gốc (seed 42, chưa resize), cạnh ngắn có trung vị 512 px, cạnh dài lớn nhất là 512 px, tỷ lệ khung hình có trung vị 1,0 (ảnh vuông) và phân vị 95 là 1,56, và chỉ 0,1% ảnh có cạnh ngắn dưới 256 px (`results/eda_sizes.json`). Sau bước resize offline (Mục 2.6), toàn bộ 101 000 ảnh được chuyển sang RGB mà không có ảnh nào lỗi (bước tạo cache dừng nếu có ảnh không chuyển được). Nhóm không thống kê chế độ màu gốc của ảnh. Kiểm tra tự động xác nhận 101 lớp, các tập rời nhau, mỗi lớp có đúng 700/50/250 ảnh, và kích thước chính thức 75 750 (train, trước khi tách validation) và 25 250 (test).



Hình 2.1: Số ảnh mỗi lớp của ba tập. Mọi lớp có cùng số ảnh trong từng tập (cực tiểu bằng cực đại), nên không cần reweighting.



Hình 2.2: Mẫu ảnh huấn luyện Food-101: sáu lớp được chọn theo quy tắc cố định (lớp thứ $0, 17, 34, 51, 68, 85$), sáu ảnh mỗi lớp.



Hình 2.3: Ảnh gốc (cột đầu) và năm lần augmentation (RandomResizedCrop, lật ngang, RandAugment) của cùng một ảnh, dùng chính seed theo mẫu của bộ huấn luyện.

2.5 Chất lượng nhãn

Nhãn huấn luyện được gán tự động theo phân loại món của Foodspotting và không được rà soát thủ công [1], vì vậy độ chính xác tối đa đạt được bị giới hạn bởi nhiễu nhãn. Hình 2.2 đã cho một ví dụ: ảnh thứ hai của lớp *apple pie* là ảnh chụp các máy tính bảng đang phát video, không phải món ăn. Ở phần phân tích lỗi, nhóm minh họa hiện tượng này bằng các ảnh huấn luyện có

loss cao nhất (Hình 6.8).

2.6 Tiền xử lý, tăng cường dữ liệu và bộ nạp dữ liệu

Mục này mô tả quy trình dữ liệu ở mức khái niệm; giá trị chính xác của từng tham số nằm ở Bảng 5.3 (Chương 5) và trong các tệp cấu hình.

Tiền xử lý offline. Mọi ảnh được resize một lần về cạnh ngắn 256 px (Pillow, nội suy bicubic, JPEG chất lượng 90; chỉ thu nhỏ, không phóng to) để giảm chi phí giải mã khi huấn luyện. Bước này nằm trong mã nguồn của repo (Chương 8); bản cache chỉ là cách tiết kiệm thời gian chạy lại bước đó, không đổi kết quả của bước.

Tăng cường dữ liệu khi huấn luyện. Mỗi ảnh được cắt ngẫu nhiên theo tỷ lệ diện tích 0,25–1 và tỷ lệ khung hình 3/4–4/3 rồi đưa về 224×224, lật ngang với xác suất 0,5, và áp dụng RandAugment [2] (hai phép biến đổi ngẫu nhiên, độ mạnh cố định ở mức nhẹ). Sau đó, mỗi batch được trộn bằng Mixup [18] hoặc CutMix [17] (chọn một trong hai) cùng label smoothing. Mục đích là giảm overfitting trên tập huấn luyện 70 700 ảnh có nhiễu nhãn, đặc biệt với các chế độ huấn luyện nhiều tham số. Cùng một công thức được dùng cho cả hai họ kiến trúc.

Khi đánh giá. Ảnh được resize về cạnh ngắn 256, cắt giữa 224×224 và chuẩn hóa mean/std ImageNet, giống nhau cho cả bốn mô hình; không có augmentation.

Bộ nạp dữ liệu. Batch 64, xáo trộn mỗi epoch, hai worker mỗi tiến trình huấn luyện. Để bộ nạp không là nút cổ chai, các phép biến đổi trên CPU chỉ cho ra tensor kiểu uint8 và việc đổi sang số thực cùng chuẩn hóa mean/std được chuyển sang GPU. Dù vậy, với các chế độ freeze (GPU xử lý nhanh) bộ nạp vẫn là nút cổ chai, nên thời gian thực tế mỗi epoch gần như không phản ánh được lợi ích tính toán của freeze; vì vậy chi phí được so sánh bằng phép đo kiểm soát chỉ trên GPU (Mục 5.8). Augmentation của từng ảnh được sinh từ một seed phụ thuộc *(seed lần chạy, epoch, chỉ số ảnh)*, nên ảnh sau augmentation không phụ thuộc số worker hay máy chạy.

Chương 3

CÁC MÔ HÌNH CNN VÀ TRANSFORMER

Hai họ kiến trúc được so sánh ở cùng quy mô tham số và chi phí tính toán, đều dùng trọng số pretrained ImageNet-1k từ thư viện `timm` [15], tức cùng dữ liệu pretrain. Cặp 1 là phần cam kết; cặp 2 là phần mở rộng, chạy nếu còn tài nguyên tính toán.

Bảng 3.1: Các mô hình được so sánh (tham số tính với đầu ra 101 lớp).

Họ	Mô hình	Trọng số (timm)	Tham số (M)	GMACs
CNN (cặp 1)	ResNet-50 [7]	`resnet50.a1_in1k` [16]	23,71	4,110
Transformer (cặp 1)	DeiT-Small/16 [14]	`deit_small_patch16_224.fb_in1k`	21,70	4,608
CNN (cặp 2)	ConvNeXt-Tiny [10]	`convnext_tiny.fb_in1k`	27,90	4,470
Transformer (cặp 2)	Swin-Tiny [9]	`swin_tiny_patch4_window7_224.ms_in1k`	27,60	4,509

Quy ước đo chi phí tính toán. GMACs được đo bằng *một* công cụ và *một* quy ước cho cả bốn kiến trúc: `fvcore` (`FlopCountAnalysis`), một lượt forward với đầu vào $1 \times 3 \times 224 \times 224$ và đầu ra 101 lớp, trong cùng lần chạy `cost_bench.py` với các số đo chi phí khác (Tesla T4, PyTorch 2.11.0+cu128; số liệu trong `results/cost_infer.json`). Một phép nhân-cộng (MAC) được tính là một đơn vị; chỉ đếm phép nhân ma trận và tích chập, không đếm phép element-wise, softmax và chuẩn hóa. Vì `fvcore` bỏ qua các phép nhân ma trận của attention khi attention được hợp nhất (`scaled_dot_product_attention`), nhóm tắt `fused attention` của `timm` khi đo GMACs để mọi phép nhân ma trận của attention đều được đếm (lệnh `python -m a1lib.cost_bench --gmacs-only`); độ trễ, thông lượng và bộ nhớ vẫn đo với cấu hình suy luận mặc định. Phần

attention của DeiT-S khoảng 0,36 GMACs, khớp với ước tính giải tích $2 \times 197^2 \times 384 \times 12$. Kết quả: ResNet-50 và DeiT-S chênh khoảng 12%, còn Swin-T thấp hơn DeiT-S khoảng 2%. Các con số "FLOPs" trong các bài báo gốc thực chất là MACs và chỉ dùng để kiểm tra thô.

3.1 Kiến trúc CNN

ResNet-50 [7]. Một lớp tích chập 7×7 bước 2 cùng max-pooling, tiếp theo là bốn stage gồm 3, 4, 6 và 3 khối bottleneck (1×1, 3×3, 1×1) với kết nối tắt cộng đầu vào vào đầu ra, rồi global average pooling và một lớp tuyến tính. Chuẩn hóa dùng BatchNorm. Kết nối tắt giúp gradient lan truyền qua mạng sâu; phần lớn tham số nằm ở stage cuối (`layer4`). Trọng số `a1_in1k` đến từ công thức huấn luyện "ResNet strikes back" [16].

ConvNeXt-Tiny [10] (cặp 2). Một CNN được thiết kế lại theo các thực hành của Transformer: lớp "patchify" 4×4 bước 4, bốn stage gồm 3, 3, 9 và 3 khối, mỗi khối dùng tích chập depthwise 7×7, LayerNorm, khối MLP dạng inverted bottleneck và kích hoạt GELU. Mục đích của việc đưa vào là tách ảnh hưởng của họ kiến trúc khỏi ảnh hưởng của đời thiết kế.

3.2 Kiến trúc Transformer

DeiT-Small [3, 14]. Ảnh 224×224 được cắt thành 196 patch 16×16, mỗi patch được chiếu tuyến tính thành vector 384 chiều, thêm token CLS và embedding vị trí. Chuỗi 197 token đi qua 12 khối Transformer (6 đầu chú ý, LayerNorm, MLP); đầu ra của token CLS đi vào lớp phân loại. Chú ý toàn cục có từ khối đầu tiên và chi phí tính chú ý tăng theo bình phương số token. Trọng số `fb_in1k` được huấn luyện chỉ trên ImageNet-1k với augmentation và regularization mạnh.

Swin-Tiny [9] (cặp 2). Transformer phân cấp: patch 4×4, bốn stage gồm 2, 2, 6 và 2 khối, chú ý được tính trong cửa sổ cục bộ 7×7 và cửa sổ được dịch xen kẽ giữa các khối để thông tin lan sang cửa sổ lân cận; giữa các stage có bước gộp patch làm giảm độ phân giải như CNN. Swin vì thế đưa lại tính cục bộ và cấu trúc phân cấp vào Transformer.

3.3 Lý do chọn và inductive bias

Cặp 1 (ResNet-50 và DeiT-S) có số tham số xấp xỉ nhau (Bảng 3.1) và đều là mô hình "tiêu chuẩn" của mỗi họ, nên là phép so sánh sạch nhất giữa tích chập và chú ý toàn cục. Cặp 2 thêm hai mô hình hiện đại hơn có cấu trúc gần nhau (phân cấp bốn stage), để xem kết luận có phụ thuộc vào đời thiết kế hay không. Kỳ vọng ban đầu, sẽ được kiểm chứng bằng số liệu ở Chương 7,

là CNN nhờ tính cục bộ và chia sẻ trọng số cần ít dữ liệu hơn để học tốt từ đầu, còn ViT ít giả định hơn về cấu trúc ảnh nên phụ thuộc nhiều hơn vào dữ liệu hoặc pretrain.

3.4 Các chế độ đóng băng

Bảng 3.2 cho phần được huấn luyện của từng chế độ; tỷ lệ tham số trainable được đo trên mô hình thực (đầu ra 101 lớp). Với các kiến trúc phân cấp (ResNet, ConvNeXt, Swin), freeze một phần nghĩa là chỉ huấn luyện stage cuối cùng cùng phần đầu ra; với DeiT là nửa sau của các block cùng chuẩn hóa cuối và phần đầu ra. Các tỷ lệ này khác nhau giữa các kiến trúc nên được ghi rõ ở mọi bảng kết quả.

Bảng 3.2: Phần được huấn luyện ở mỗi chế độ và tỷ lệ tham số trainable (đo trên mô hình).

Chế độ	Phần được huấn luyện	ResNet-50	DeiT-S	ConvNeXt-T	Swin-T
Freeze toàn bộ backbone	Chỉ lớp phân loại	0,9%	0,2%	0,3%	0,3%
Freeze một phần	Stage cuối + đầu ra; DeiT: block 7–12 + norm + đầu ra	64,0%	49,2%	55,7%	56,0%
Full fine-tuning	Toàn bộ	100%	100%	100%	100%
From-scratch	Toàn bộ, khởi tạo ngẫu nhiên	100%	100%	100%	100%

Phần bị đóng băng được đặt ở chế độ eval, nên BatchNorm không cập nhật thống kê.

3.5 Lưu ý khi diễn giải

Không dùng ViT-Base/16 (khoảng 86M tham số) vì so sánh với ResNet-50 sẽ lẫn lộn ảnh hưởng của kiến trúc với ảnh hưởng của quy mô mô hình, và chi phí huấn luyện cao hơn nhiều. Các trọng số pretrained của hai họ được huấn luyện bằng các công thức khác nhau (ví dụ ResNet-50 "a1" dùng công thức huấn luyện mạnh [16]), nên một phần khác biệt giữa hai họ có thể đến từ công thức pretrain chứ không chỉ từ kiến trúc. Cặp ConvNeXt-Tiny/Swin-Tiny được đưa vào để kiểm tra khác biệt do họ kiến trúc hay do đời thiết kế.

Chương 4

CHIẾN LƯỢC HUẤN LUYỆN

Chương này mô tả các chiến lược huấn luyện được so sánh và những gì chúng khác nhau về tham số được cập nhật và chi phí. Các nhận định về hiệu quả là *kỳ vọng* dựa trên kiến thức chung, được kiểm chứng bằng số liệu ở Chương 7; chỉ các số đo chi phí (đo trên Tesla T4, Mục 5.8) là kết quả thực.

4.1 Huấn luyện từ đầu (from-scratch)

Mô hình được khởi tạo ngẫu nhiên và mọi tham số được học chỉ từ tập huấn luyện của nhóm (70 700 ảnh). Mô hình phải học cả các đặc trưng cấp thấp (cạnh, kết cấu) lẫn đặc trưng cấp cao của món ăn từ lượng dữ liệu nhỏ hơn nhiều so với ImageNet-1k (khoảng 1,28 triệu ảnh), nên thường cần nhiều epoch hơn, augmentation mạnh hơn và có nguy cơ overfitting cao hơn. Kỳ vọng là kiến trúc có inductive bias mạnh (CNN) học tốt hơn từ ít dữ liệu so với kiến trúc ít giả định (ViT); nhóm kiểm chứng bằng from-scratch 40 epoch cho cả hai họ và ablation theo kích thước dữ liệu. Đây cũng là chế độ đắt nhất: 40 epoch cho mỗi mô hình so với 10 epoch ở các chế độ pretrained.

4.2 Sử dụng mô hình pretrained

Transfer learning khởi tạo mô hình từ trọng số học ở tác vụ nguồn (ImageNet-1k) và điều chỉnh cho tác vụ đích (Food-101, 101 lớp). Ba chiến lược sau khác nhau ở phần trọng số được cập nhật (Bảng 3.2).

4.2.1 Đóng băng toàn bộ backbone (linear probing)

Mọi trọng số của backbone giữ nguyên, chỉ lớp phân loại 101 lớp được huấn luyện từ khởi tạo ngẫu nhiên; phần đóng băng được đặt ở chế độ eval nên BatchNorm không cập nhật thống kê.

Chiến lược này ít tham số cần học nhất (dưới 1%), ít nguy cơ overfitting và rẻ nhất về bộ nhớ, nhưng chất lượng bị giới hạn bởi mức độ phù hợp của đặc trưng ImageNet với tập đích.

4.2.2 Fine-tune một phần

Các tầng đầu (đặc trưng tổng quát) được đóng băng; stage cuối cùng của CNN hoặc nửa sau các khối của DeiT, cùng lớp phân loại, được huấn luyện. Các tầng sau thường mang đặc trưng riêng cho tác vụ nguồn hơn nên là phần có lợi nhất khi điều chỉnh, còn tầng đầu giữ nguyên giúp giảm tham số cần cập nhật và nguy cơ overfitting. Vị trí cắt là lựa chọn thiết kế (một mức duy nhất cho mỗi kiến trúc) và không được tối ưu hóa.

4.2.3 Fine-tune toàn bộ

Khởi tạo từ trọng số pretrained rồi cập nhật mọi tham số, với learning rate đỉnh nhỏ để không phá đặc trưng đã học. Đây là chiến lược linh hoạt nhất và thường cho độ chính xác cao nhất khi có đủ dữ liệu, nhưng tốn bộ nhớ nhất và có nguy cơ overfitting hoặc quên bớt đặc trưng tổng quát khi tập đích nhỏ hoặc nhiễu.

4.3 So sánh các chiến lược

4.3.1 Tham số được cập nhật

Tỷ lệ tham số trainable của từng chế độ và kiến trúc nằm ở Bảng 3.2: dưới 1% khi freeze toàn bộ, khoảng một nửa đến hai phần ba khi freeze một phần, và 100% khi fine-tune toàn bộ hoặc huấn luyện từ đầu. Do cách cắt khác nhau, cùng một chế độ "freeze một phần" không ứng với cùng tỷ lệ giữa các kiến trúc (64,0% ở ResNet-50 so với 49,2% ở DeiT-S), nên khi so sánh chế độ này giữa hai họ cần đọc kèm tỷ lệ.

4.3.2 Chi phí tính toán (đo kiểm soát)

Giảm số tham số trainable không giảm tỷ lệ thuận chi phí, vì lượt forward qua backbone vẫn cần cho mọi bước; freeze chỉ bỏ phần backward và trạng thái optimizer của phần bị đóng băng, và cho phép không lưu activation của nó. Bảng 4.1 cho thấy hệ quả đo được trên Tesla T4 (batch 64, AMP fp16, batch tổng hợp trên GPU; chi tiết ở Mục 5.8).

Bảng 4.1: Chi phí tương đối của các chế độ so với full fine-tuning (Tesla T4, AMP fp16, batch 64, batch tổng hợp trên GPU; đo bằng `cost_bench.py`).

Kiến trúc	Chế độ	s/epoch	Nhanh hơn full	Bộ nhớ (MB)	Bộ nhớ so với full
ResNet-50	Linear probing	70,5	2,8×	470	15%
ResNet-50	Freeze một phần	86,4	2,3×	646	20%
ResNet-50	Full fine-tuning	195,0	1,0×	3 161	100%
DeiT-S	Linear probing	58,3	3,3×	287	11%
DeiT-S	Freeze một phần	122,8	1,6×	1 336	53%
DeiT-S	Full fine-tuning	194,4	1,0×	2 502	100%
ConvNeXt-T	Linear probing	102,4	3,0×	680	16%
ConvNeXt-T	Freeze một phần	126,2	2,4×	861	20%
ConvNeXt-T	Full fine-tuning	302,6	1,0×	4 353	100%
Swin-T	Linear probing	152,6	2,7×	921	19%
Swin-T	Freeze một phần	177,5	2,3×	1 099	23%
Swin-T	Full fine-tuning	406,1	1,0×	4 832	100%

Kết quả cho thấy lợi ích của freeze về bộ nhớ lớn hơn lợi ích về thời gian, và lợi ích phụ thuộc kiến trúc: freeze một phần ở DeiT-S (nửa sau các khối) tiết kiệm ít hơn ở ResNet-50 (chỉ stage cuối). Ngoài ra, trong các lần chạy thật, nút cổ chai ở bộ nạp dữ liệu làm các chế độ freeze gần như không nhanh hơn về thời gian thực (Mục 5.8), nên lợi ích về thời gian chỉ thấy rõ khi dữ liệu được cấp đủ nhanh.

4.3.3 Ưu điểm và hạn chế

Bảng 4.2: Tóm tắt kỳ vọng về các chiến lược (sẽ được kiểm chứng ở Chương 7).

Chiến lược	Ưu điểm kỳ vọng	Hạn chế kỳ vọng
From-scratch	Không phụ thuộc miền dữ liệu của tác vụ nguồn; kiến trúc tự do	Đắt nhất; cần nhiều dữ liệu và epoch; ViT thường kém hơn CNN khi dữ liệu ít
Freeze toàn bộ	Rẻ nhất; ít overfitting; huấn luyện nhanh hội tụ	Giới hạn bởi đặc trưng ImageNet; thường kém nhất trong các chế độ pretrained
Freeze một phần	Cân bằng giữa chi phí và khả năng thích ứng	Vị trí cắt là lựa chọn cố định; lợi ích phụ thuộc kiến trúc
Fine-tune toàn bộ	Thường cho độ chính xác cao nhất	Tốn bộ nhớ nhất; nhạy với learning rate; có thể quên đặc trưng tổng quát

Chương 5

THIẾT KẾ THỰC NGHIỆM

5.1 Biến nghiên cứu

Bảng 5.1: Biến độc lập, biến phụ thuộc và các yếu tố được giữ cố định.

Loại biến	Nội dung
Độc lập	Họ kiến trúc (CNN / Transformer); khởi tạo (ngẫu nhiên / pretrained ImageNet-1k); chế độ huấn luyện (freeze toàn bộ / freeze một phần / full fine-tuning); tỷ lệ dữ liệu huấn luyện (10, 25, 50, 100%) trong ablation; seed
Phụ thuộc	Top-1, top-5, macro-F1 trên test; số tham số trainable; GMACs; thời gian huấn luyện; thông lượng và độ trễ suy luận; bộ nhớ GPU đỉnh; ECE, NLL, độ bền vững
Giữ cố định	Split dữ liệu; công thức augmentation và regularization; bộ tối ưu; hình dạng lịch learning rate; batch; số epoch theo chế độ; phép biến đổi khi đánh giá; quy ước đo chi phí (Mục 1.4)

5.2 Ma trận thí nghiệm

Với mỗi mô hình ở cặp 1, nhóm chạy bốn cấu hình trên cùng một split, tổng cộng tám cấu hình lõi (Bảng 5.2).

Bảng 5.2: Tám cấu hình lõi (cặp 1).

Mô hình	Chế độ	Khởi tạo	Phần huấn luyện	Epoch
ResNet-50, DeiT-S	From-scratch	Ngẫu nhiên	Toàn bộ	40
ResNet-50, DeiT-S	Freeze toàn bộ backbone	ImageNet-1k	Lớp phân loại	10
ResNet-50, DeiT-S	Freeze một phần	ImageNet-1k	Xem Bảng 3.2	10
ResNet-50, DeiT-S	Full fine-tuning	ImageNet-1k	Toàn bộ	10

5.3 Công thức huấn luyện chi tiết

Một công thức duy nhất được dùng cho mọi mô hình và chế độ; các giá trị này nằm trong các tệp cấu hình YAML được commit cùng mã nguồn (Phụ lục A).

Bảng 5.3: Công thức huấn luyện chính xác.

Hạng mục	Giá trị chính xác
Phần mềm	Python 3.13, PyTorch 2.11.0, torchvision 0.26.0, timm 1.0.30; phiên bản Pillow và NumPy được ghi vào `results/*.json` của mỗi lần chạy.
Resize offline	Pillow: cạnh ngắn → 256 px (chỉ thu nhỏ), nội suy bicubic, JPEG chất lượng 90.
Biến đổi khi train	(1) `RandomResizedCrop(224, scale=(0.25, 1.0), ratio=(3/4, 4/3))`, nội suy bilinear (`scale` khác mặc định (0,08, 1)); (2) `RandomHorizontalFlip(p=0.5)`; (3) `RandAugment(num_ops=2, magnitude=9, num_magnitude_bins=31, interpolation=NEAREST, fill=None)` của torchvision [2]; (4) `PILToTensor` (uint8); (5) trên GPU: chuẩn hóa mean (0,485; 0,456; 0,406), std (0,229; 0,224; 0,225); (6) Mixup/CutMix.
Độ mạnh RandAugment	Mỗi ảnh nhận 2 phép chọn ngẫu nhiên có hoàn lại trong 14 phép, luôn được áp dụng, dấu của phép có dấu chọn ngẫu nhiên. Chỉ số `magnitude=9` trên 31 mức tương ứng $9/30 = 30\%$ biên độ tối đa của mỗi phép (ví dụ xoay $\pm 9°$, shear $\pm 0{,}09$, độ sáng/màu/tương phản $\pm 0{,}27$). Mức này *nhẹ hơn nhiều* so với `rand-m9` trong timm/DeiT ($9/10 = 90\%$ biên độ). Cấu hình được giữ cố định cho mọi mô hình và không tinh chỉnh.

Hạng mục	Giá trị chính xác
Mixup/CutMix	`timm.data.Mixup(mixup_alpha=0.8, cutmix_alpha=1.0, cutmix_minmax=None, prob=1.0, switch_prob=0.5, mode='batch', correct_lam=True, label_smoothing=0.1, num_classes=101)` [17, 18]. Mỗi batch (`prob=1.0`) nhận *đúng một* trong hai: Mixup với xác suất 0,5 ($\lambda \sim \text{Beta}(0{,}8; 0{,}8)$) hoặc CutMix với xác suất 0,5 ($\lambda \sim \text{Beta}(1; 1)$, sau đó λ được hiệu chỉnh theo diện tích hộp thực tế sau khi cắt biên); hai phương pháp loại trừ nhau trong một batch. Mỗi mẫu được ghép với mẫu đối xứng trong batch (đảo thứ tự batch); batch 64 (chẵn), `drop_last=True`. Việc chọn phương pháp và λ dùng bộ sinh số ngẫu nhiên toàn cục của NumPy trong tiến trình chính, nên được seed và được lưu/khôi phục cùng checkpoint.
Nhãn mềm và loss	Nhãn đích $y = \lambda\, y_s(a) + (1-\lambda)\, y_s(b)$ với $y_s(c) = (1-\varepsilon)\,\text{onehot}(c) + \varepsilon/101$, $\varepsilon = 0{,}1$ [13]. Label smoothing được áp dụng **một lần duy nhất, bên trong Mixup**. Loss huấn luyện là soft-target cross-entropy $-\sum_k y_k \log \text{softmax}(z)_k$ tính ở fp32 và **không** truyền thêm tham số label smoothing vào hàm loss. Khi đánh giá (validation/test) dùng cross-entropy với nhãn cứng, không smoothing; NLL và ECE tính trên nhãn cứng.
Biến đổi khi đánh giá	`Resize(256)` (bilinear) → `CenterCrop(224)` → chuẩn hóa mean/std ImageNet; cùng một phép cho cả bốn mô hình.
Tối ưu hóa	AdamW [11] ($\beta = (0{,}9; 0{,}999)$, $\epsilon = 10^{-8}$), weight decay 0,05 chỉ cho tham số từ 2 chiều trở lên (không áp dụng cho bias và tham số chuẩn hóa); không gradient clipping; không EMA; `drop_rate=0` và `drop_path_rate=0` được đặt tường minh cho mọi mô hình (không dùng giá trị mặc định riêng của từng mô hình). Mixed precision fp16 với GradScaler động.
Lịch learning rate	Theo từng iteration: warmup tuyến tính từ 1% LR đỉnh lên LR đỉnh trong 5% tổng số bước đầu (0,5 epoch với 10 epoch; 2 epoch với 40 epoch), sau đó giảm cosine tới 1% LR đỉnh ở bước cuối. Cùng hình dạng lịch cho lượt quét LR và các lần chạy chính.
Batch và dữ liệu	Batch 64, `drop_last=True`, 2 worker, xáo trộn mỗi epoch. Thứ tự dữ liệu và ngẫu nhiên của augmentation sinh từ generator seed theo (seed, epoch).
Seed và tái lập	Seed 0/1/2 (lượt quét LR dùng seed 0). Trạng thái RNG của Python, NumPy, PyTorch, CUDA cùng GradScaler, optimizer, scheduler được lưu trong checkpoint mỗi epoch; khôi phục RNG cho kết quả trùng khớp từng bit trên một mô hình thử khi bật chế độ xác định. Với huấn luyện thật, `cudnn.benchmark` được bật để nhanh hơn nên kết quả không bảo đảm trùng từng bit giữa các lần chạy hoặc giữa các loại GPU; sai khác được ước lượng từ 3 seed.

Epoch báo cáo	Epoch cuối cùng; không chọn checkpoint theo validation.

5.4 Quét learning rate

Learning rate đỉnh được chọn bằng một lượt quét nhỏ chạy trước các lần chạy chính (Bảng 5.4). Mỗi mức LR chạy 3 epoch trên 20% tập train (140 ảnh mỗi lớp, cân bằng lớp, cố định bằng seed 42), seed 0, và đánh giá trên toàn bộ validation (5 050 ảnh).

Bảng 5.4: Lưới learning rate đỉnh cho mỗi mô hình.

Nhóm chế độ	Lưới LR	Áp dụng cho
Freeze toàn bộ backbone	$\{3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}\}$	Linear probing
Fine-tune (quét bằng full fine-tuning)	$\{3 \cdot 10^{-5}, 10^{-4}, 3 \cdot 10^{-4}\}$	Freeze một phần và full fine-tuning dùng chung một LR
From-scratch	$\{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}\}$	From-scratch

- **Tiêu chí chọn:** top-1 trên validation ở *epoch cuối (epoch 3)* của lượt quét; nếu hòa, chọn NLL validation thấp hơn.
- **Quy tắc biên:** nếu LR tốt nhất nằm ở biên lưới, mở rộng thêm một mức (nhân hoặc chia 3) về phía đó và chạy lại riêng mức đó.
- **Hạn chế:** lượt quét ngắn (3 epoch, 20% dữ liệu) có thể thiên về LR lớn hơn mức tối ưu của lần chạy dài, nhất là với from-scratch; kết quả được xem là lựa chọn thực dụng, không phải tối ưu hóa toàn cục.

5.5 Các thí nghiệm và ngân sách tính toán

Số epoch được cố định từ trước vì lịch cosine phụ thuộc tổng số epoch. Bảng 5.5 liệt kê các thí nghiệm theo thứ tự ưu tiên cùng chi phí ước tính trước khi chạy (GPU-giờ Tesla T4, từ các phép đo tốc độ sơ bộ). Chi phí thực tế được nêu ngay sau bảng. Phần cam kết khoảng 24,6 GPU-giờ (dự trù thêm khoảng 25% cho khởi động lại phiên, tức khoảng 30 giờ thực tế); các mức mở rộng chỉ chạy khi còn hạn mức GPU, theo đúng thứ tự trong bảng, và mỗi mức độc lập nên việc dừng sớm vẫn cho báo cáo hoàn chỉnh.

Bảng 5.5: Thí nghiệm theo thứ tự ưu tiên và chi phí ước tính.

Mức	Thí nghiệm	Trả lời câu hỏi	GPU-giờ
Cam kết	Quét LR (chạy đầu tiên)	Công bằng giữa kiến trúc	0,8
Cam kết	8 cấu hình lõi, 1 seed (6 pretrained × 10 epoch + 2 scratch × 40 epoch)	RQ1, RQ2, RQ3	9,6
Cam kết	Ablation theo kích thước dữ liệu: 10%, 25%, 50% (cân bằng lớp, lồng nhau) × {scratch 40 epoch, full fine-tune 10 epoch} × 2 mô hình	RQ2	6,4
Cam kết	Thêm 2 seed cho 6 cấu hình pretrained (tổng 3 seed)	Phương sai, độ tin cậy	7,8
Mở rộng 1	ConvNeXt-T vs Swin-T, 3 chế độ pretrained	Họ kiến trúc hay đời thiết kế	4,7
Mở rộng 2	DeiT-S from-scratch 100 epoch (đối chứng bất đối xứng)	Khoảng cách của ViT có do huấn luyện quá ngắn không	7,3
Mở rộng 3	ConvNeXt-T và Swin-T from-scratch 40 epoch	Lặp lại so sánh scratch với cặp 2	8,4

Sau khi hoàn tất, tổng thời gian huấn luyện cộng validation của 88 lần chạy là khoảng 52 GPU-giờ: quét LR 2,9 giờ (47 lần chạy, gồm các mức LR mở rộng), các cấu hình lõi và seed 21,0 giờ (kế hoạch 17,4), ablation 7,1 giờ (kế hoạch 6,4) và các lần chạy mở rộng 21,1 giờ (kế hoạch 20,4). Các thí nghiệm bổ sung sau giao thức (Mục 5.6, 25 lần chạy) tốn thêm khoảng 12,0 giờ, nâng tổng lên khoảng 64 GPU-giờ. Chi phí thực tế cao hơn kế hoạch khoảng 7 giờ: lượt quét LR tốn 2,9 giờ so với 0,8 giờ dự trù (trong đó 0,7 giờ là 11 mức LR mở rộng; phần còn lại nhóm không tách riêng), các cấu hình lõi và seed vượt kế hoạch 3,6 giờ, còn ablation và các lần chạy mở rộng mỗi phần vượt khoảng 0,7 giờ. Dự trù dựa trên tốc độ chỉ-GPU, trong khi thời gian thực mỗi epoch bị giới hạn bởi bộ nạp dữ liệu (Mục 5.8).

5.6 Thí nghiệm bổ sung sau giao thức

Sau khi 88 lần chạy của giao thức hoàn tất và các kết quả chính đã được xem, nhóm bổ sung một số thí nghiệm nhằm kiểm tra các giả thuyết nảy sinh từ chính kết quả đó. Đây là *phần bổ sung sau khi biết kết quả*, không thuộc giao thức đã cam kết, nên nhóm đặt ra quy tắc trước khi chạy (được ghi trong mã nguồn) và không thay thế bất kỳ kết quả nào của giao thức: chúng chỉ được báo cáo bên cạnh.

- **Tiếp tục quét LR:** với các nhóm mà LR được chọn nằm ở biên lưới đã mở rộng (ResNet-50 linear probing và full fine-tuning, DeiT-S linear probing) thêm một mức LR (nhân 3), cùng thiết lập quét ngắn (3 epoch, 20% dữ liệu). Nếu mức mới cao hơn LR đã chọn ít nhất 0,5 điểm top-1 validation thì chạy *một* lần 10 epoch (seed 0) ở LR đó; kết quả giao thức không bị thay.
- **LR riêng cho freeze một phần:** giao thức dùng chung LR của full fine-tuning cho hai chế độ; nhóm quét riêng cho freeze một phần của hai mô hình và áp dụng cùng quy tắc.
- **Nguồn gốc của sự thiếu tự tin:** full fine-tuning 10 epoch (ResNet-50 và DeiT-S) không dùng Mixup/CutMix (giữ label smoothing 0,1), và không dùng cả hai (nhãn cứng).
- **Layer-wise LR decay:** DeiT-S full fine-tuning với hệ số giảm 0,75 cho mỗi block (block cuối nhận LR đỉnh; embedding nhận $0{,}75^{13}$ lần), cùng LR đỉnh như giao thức.
- **Phương sai của độ bền vững:** đo lại độ bền vững trên seed 1 và 2 của sáu cấu hình pretrained (không huấn luyện thêm).

Mọi lần chạy bổ sung chỉ dùng seed 0 (trừ phép đo độ bền vững và cặp ResNet-50 freeze một phần ở LR riêng / full fine-tuning ở LR giao thức, được chạy thêm đến 5 seed mỗi bên vì chênh lệch giữa chúng nhỏ và quyết định kết luận về RQ1; số seed được ấn định trước khi chạy và kết quả được báo cáo dù theo hướng nào), nên chỉ những khác biệt lớn hơn rõ rệt so với độ lệch chuẩn giữa seed của giao thức (0,07–0,24 điểm) được diễn giải.

5.7 Giao thức đánh giá và phân tích

Chỉ số. Top-1, top-5 và macro-F1 trên tập test; số tham số trainable và GMACs; thời gian mỗi epoch và tổng thời gian huấn luyện; thông lượng và độ trễ suy luận; bộ nhớ GPU đỉnh. Cách đo chi phí được định nghĩa ở Mục 5.8.

Phân tích bổ sung (chỉ cần suy luận). Sau các thí nghiệm lõi, mỗi lần chạy đã lưu trọng số cuối, logits validation/test và lịch sử huấn luyện, nên các phân tích sau không cần huấn luyện thêm: (i) ma trận nhầm lẫn và phân tích lỗi, kèm các ảnh huấn luyện có loss cao nhất để minh họa nhiễu nhãn; (ii) hiệu chỉnh độ tin cậy [6]: ECE, NLL, Brier, biểu đồ reliability và temperature scaling fit trên validation; (iii) ensemble CNN+ViT so với mô hình đơn và so với ensemble cùng kiến trúc khác seed; (iv) độ bền vững trước nhiễu Gauss, làm mờ, nén JPEG, đổi tương phản, che khuất và xáo trộn patch [8] trên tập con test 50 ảnh mỗi lớp (sinh bằng seed cố định, không phân phối lại); (v) diễn giải định tính bằng Grad-CAM [12] cho CNN và bản đồ chú ý hoặc Grad-CAM trên token cho ViT trên khoảng 12 ảnh chọn trước, dùng như minh họa chứ không phải bằng chứng định lượng.

Về mất cân bằng lớp. Food-101 cân bằng tuyệt đối (700/50/250 ảnh mỗi lớp trong split của nhóm), nên không dùng reweighting, oversampling hay focal loss; vấn đề thực tế là nhiễu nhãn ở tập train.

5.8 Cách đo chi phí

Mọi số liệu chi phí được đo bằng cùng một script, trên cùng một GPU trong cùng một phiên, và báo cáo ghi rõ loại GPU.

- **Thời gian mỗi epoch, chỉ GPU:** 70 700 ảnh chia cho tốc độ huấn luyện đo được (img/s) với batch 64, AMP fp16, AdamW, batch tổng hợp đặt sẵn trên GPU (không có bộ nạp dữ liệu). Đây là cột dùng để so sánh các kiến trúc và chế độ freeze.
- **Thời gian mỗi epoch, đo thực trong lần chạy thật (Kaggle):** thời gian thực mỗi epoch (huấn luyện cộng validation) trong lần huấn luyện thật. Cột này bị giới hạn bởi bộ nạp dữ liệu nên các chế độ freeze gần như không nhanh hơn.
- **Tổng thời gian huấn luyện:** tổng thời gian các epoch, không tính thời gian chờ giữa các phiên hoặc khởi động lại.
- **Bộ nhớ GPU đỉnh:** `torch.cuda.max_memory_allocated` (trọng số, activation, gradient, trạng thái optimizer khi huấn luyện; trọng số và activation khi suy luận).
- **Suy luận:** AMP fp16, `torch.inference_mode`, batch 1 và 128, đồng bộ CUDA quanh mỗi lần gọi; độ trễ là median (và p95) của một lần gọi, thông lượng bằng batch chia độ trễ median.

Chương 6

KẾT QUẢ THỰC NGHIỆM

Chương này trình bày kết quả của toàn bộ 88 lần chạy (8 lần chạy lõi, 12 lần chạy thêm seed, 12 lần ablation, 9 lần chạy mở rộng và 47 lần quét learning rate, tổng cộng 88 lần chạy riêng biệt). Mọi con số được sinh tự động từ `results/*.json` bằng `python -m a1lib.analyze`; các bảng ở đây là các tệp `generated/tab_*.tex` được nạp trực tiếp, không nhập tay. Các lần chạy đều thực hiện trên Tesla T4 (Kaggle), PyTorch 2.11.0, timm 1.0.30; tổng thời gian huấn luyện và validation khoảng 52 giờ GPU. Mục 6.7 báo cáo thêm 25 lần chạy bổ sung sau giao thức (10 lượt quét LR ngắn và 15 lần chạy 10 epoch, tổng cộng khoảng 12,0 giờ GPU) (Mục 5.6).

6.1 Độ chính xác và F1

Bảng 6.1: Độ chính xác trên tập test (25 250 ảnh); mean ± std trên 3 seed với cấu hình pretrained, 1 seed với from-scratch.

Họ	Kiến trúc	Chế độ	Trainable	Top-1 (%)	Top-5 (%)	Macro-F1 (%)
CNN	ResNet-50	From-scratch	100,0%	74,61	92,83	74,19
CNN	ResNet-50	Linear probing	0,9%	69,25 ± 0,24	90,42 ± 0,10	68,85 ± 0,25
CNN	ResNet-50	Freeze một phần (layer4 + đầu ra)	64,0%	84,09 ± 0,08	96,63 ± 0,06	84,06 ± 0,08
CNN	ResNet-50	Full fine-tuning	100,0%	85,99 ± 0,07	97,16 ± 0,04	85,95 ± 0,06
Transformer	DeiT-S	From-scratch	100,0%	34,72	63,49	31,94
Transformer	DeiT-S	Linear probing	0,2%	70,12 ± 0,11	90,79 ± 0,04	69,82 ± 0,14
Transformer	DeiT-S	Freeze một phần (block 7–12 + đầu ra)	49,2%	88,27 ± 0,11	97,89 ± 0,03	88,26 ± 0,11
Transformer	DeiT-S	Full fine-tuning	100,0%	87,73 ± 0,13	97,63 ± 0,07	87,71 ± 0,13

Với cả hai kiến trúc, pretraining cho khác biệt rất lớn: ResNet-50 tăng từ 74,61% (from-scratch) lên 85,99% (full fine-tuning), còn DeiT-S tăng từ 34,72% lên 87,73%. Linear probing (0,2–0,9% tham số huấn luyện được) đã đạt khoảng 69–70% ở cả hai mô hình, tức đặc trưng ImageNet-1k tách được một phần lớn các lớp món ăn mà không cần cập nhật backbone. Độ lệch chuẩn giữa 3 seed rất nhỏ (0,07–0,24 điểm; Bảng B.3 ở Phụ lục B), nên các khác biệt trên 0,5 điểm giữa các chế độ đáng tin cậy, còn khác biệt dưới khoảng 0,3 điểm thì không phân biệt được với nhiễu giữa các seed. Hai kết quả đáng chú ý: ở DeiT-S, freeze một phần ($88{,}27 \pm 0{,}11\%$) cao hơn full fine-tuning ($87{,}73 \pm 0{,}13\%$) khoảng 0,54 điểm, lớn hơn cả hai độ lệch chuẩn; ở ResNet-50 thì ngược lại, full fine-tuning hơn freeze một phần khoảng 1,9 điểm (Bảng B.4).

6.1.1 Kết quả của hai kiến trúc mở rộng

Hai kiến trúc mở rộng (ConvNeXt-T, Swin-T) chỉ chạy một seed, nên các khác biệt dưới khoảng 0,5 điểm không được diễn giải. Các lần chạy này đặt cặp CNN/ViT thứ hai bên cạnh ResNet-50/DeiT-S để kiểm tra xem kết luận có phụ thuộc vào cặp kiến trúc hay không (Mục 7.1).

Bảng 6.2: Kết quả của ConvNeXt-T và Swin-T trên tập test (1 seed; pretrained 10 epoch, from-scratch 40 epoch).

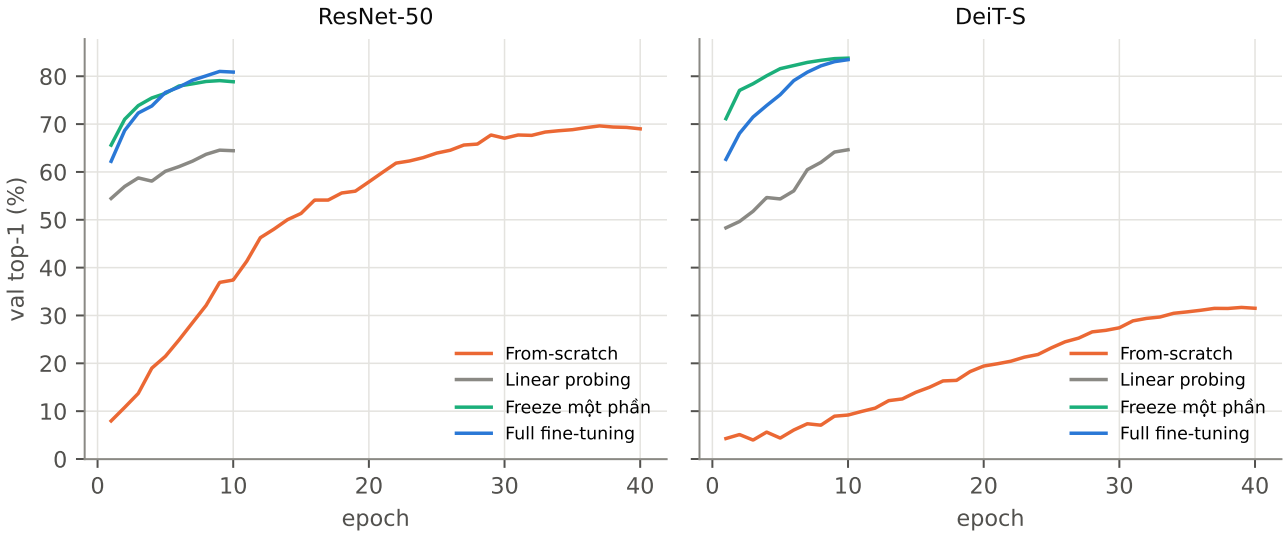
Họ	Kiến trúc	Chế độ	Trainable	Top-1 (%)	Top-5 (%)	Macro-F1 (%)
CNN	ConvNeXt-T	From-scratch	100,0%	76,82	93,53	76,63
CNN	ConvNeXt-T	Linear probing	0,3%	73,49	92,32	73,21
CNN	ConvNeXt-T	Freeze một phần (stage 4 + đầu ra)	55,7%	87,74	97,67	87,73
CNN	ConvNeXt-T	Full fine-tuning	100,0%	88,91	97,94	88,91
Transformer	Swin-T	From-scratch	100,0%	59,53	84,40	58,53
Transformer	Swin-T	Linear probing	0,3%	74,45	93,05	74,30
Transformer	Swin-T	Freeze một phần (stage 4 + đầu ra)	56,0%	87,18	97,59	87,18
Transformer	Swin-T	Full fine-tuning	100,0%	88,90	98,05	88,91

ConvNeXt-T và Swin-T cùng đạt khoảng 88,9% khi full fine-tuning, cao hơn cả hai mô hình chính; ở from-scratch 40 epoch, ConvNeXt-T (76,82%) vượt ResNet-50 (74,61%) còn Swin-T chỉ đạt 59,53%. Đối chứng huấn luyện DeiT-S từ đầu 100 epoch (Bảng 6.3) đạt 67,52%, gần gấp đôi kết quả 40 epoch (34,72%), nhưng vẫn thấp hơn ResNet-50 40 epoch.

Bảng 6.3: Các lần chạy mở rộng và đối chứng 100 epoch (1 seed).

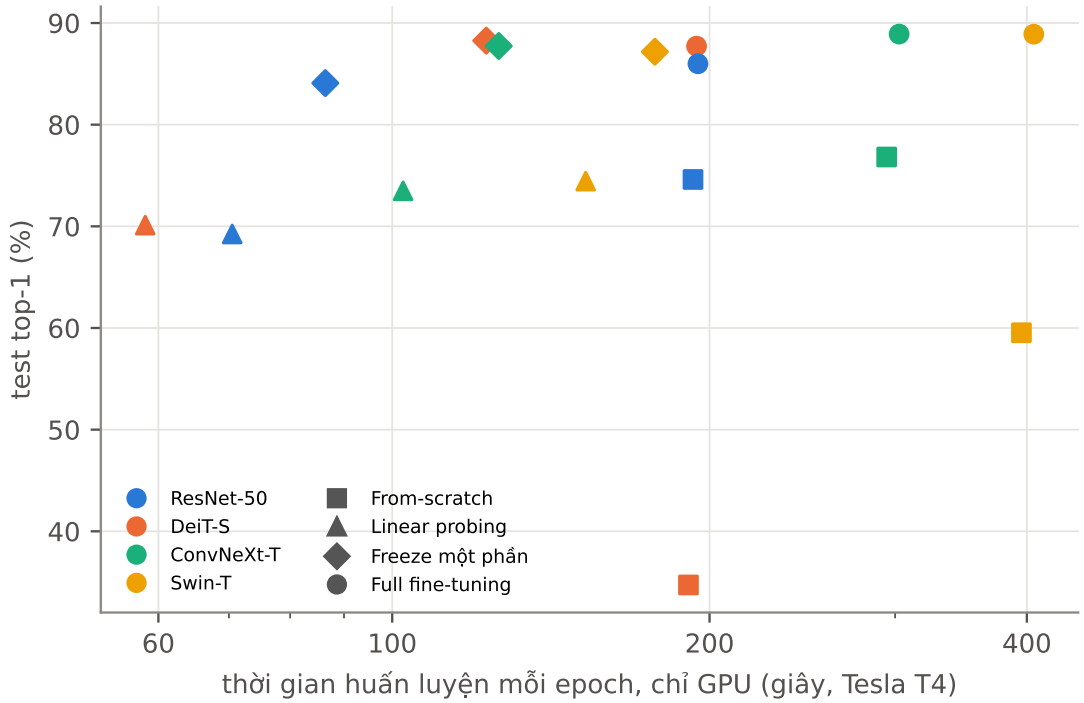
Kiến trúc	Chế độ	Epoch	Top-1 (%)	Top-5 (%)	Macro-F1 (%)	Giờ
ConvNeXt-T	Full fine-tuning	10	88,91	97,94	88,91	0,95
ConvNeXt-T	Linear probing	10	73,49	92,32	73,21	0,92
ConvNeXt-T	Freeze một phần	10	87,74	97,67	87,73	0,86
ConvNeXt-T	From-scratch	40	76,82	93,53	76,63	3,49
DeiT-S	From-scratch	100	67,52	88,26	67,08	6,74
Swin-T	Full fine-tuning	10	88,90	98,05	88,91	1,25
Swin-T	Linear probing	10	74,45	93,05	74,30	0,78
Swin-T	Freeze một phần	10	87,18	97,59	87,18	0,86
Swin-T	From-scratch	40	59,53	84,40	58,53	5,25

6.2 Diễn biến quá trình huấn luyện



Hình 6.1: Độ chính xác validation theo epoch của các chế độ huấn luyện (lần chạy seed 0).

Các chế độ pretrained hội tụ nhanh: ngay sau epoch đầu tiên, full fine-tuning và freeze một phần đã đạt khoảng 62–71% validation, và đường cong chỉ còn tăng chậm từ epoch 5. Ở ResNet-50, from-scratch đạt 67,0% validation ở epoch 30 và 69,0% ở epoch 40; nó vượt linear probing (64,4% ở cuối) từ khoảng epoch 26 nhưng vẫn thấp hơn hai chế độ fine-tune (80,9% và 78,9% ở epoch 10). Ở DeiT-S from-scratch, đường cong vẫn còn đi lên khi lịch cosine kết thúc (khoảng 31%), phù hợp với việc 40 epoch chưa đủ cho ViT khởi tạo ngẫu nhiên. Hình 6.2 đặt độ chính xác cạnh chi phí huấn luyện chỉ GPU.



Hình 6.2: Độ chính xác test theo thời gian huấn luyện mỗi epoch chỉ GPU.

6.3 Phân tích lỗi

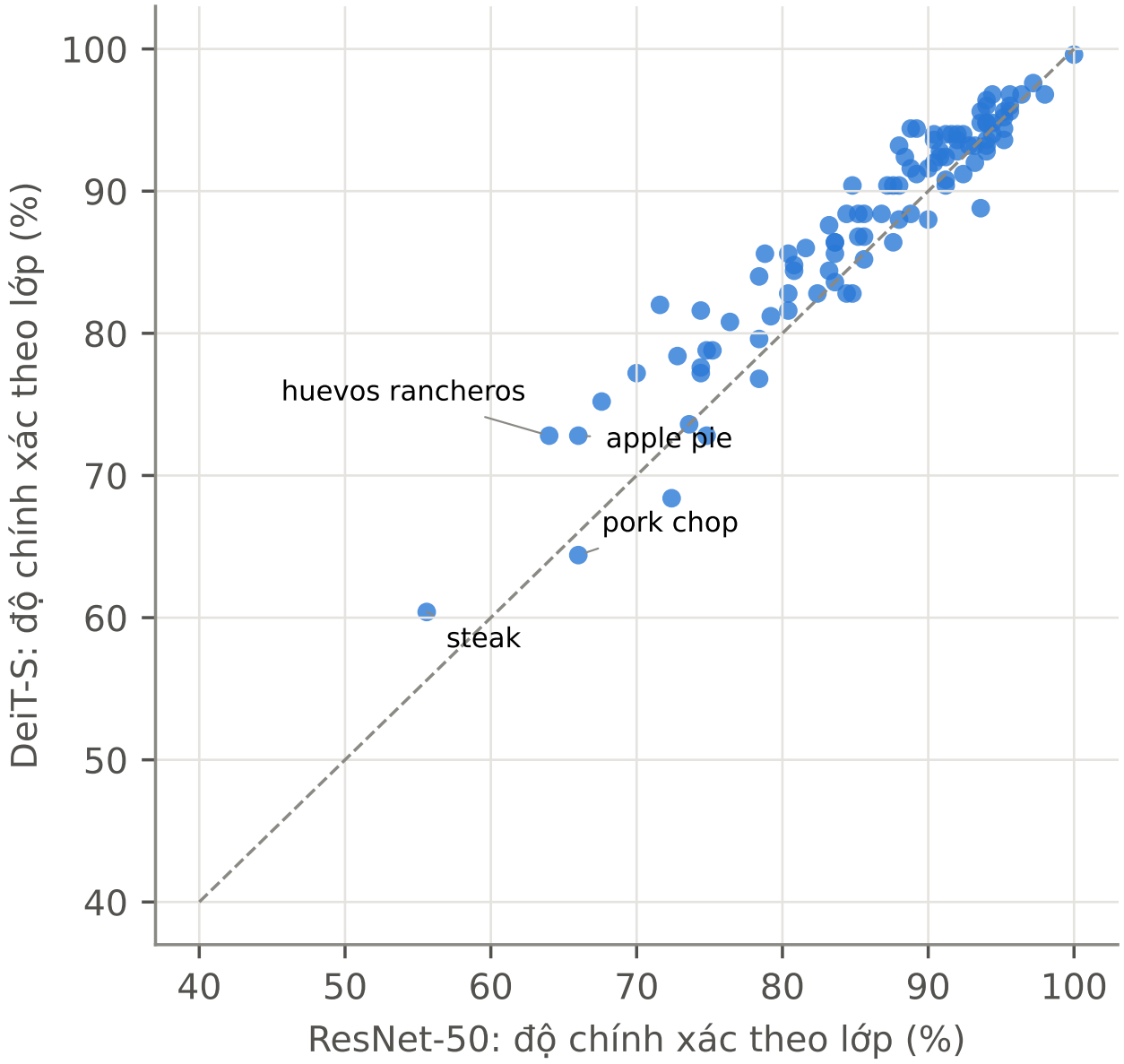
Các cặp lớp bị nhầm nhiều nhất đều là các món thực sự giống nhau về hình thức hoặc nguyên liệu, và gần như trùng khớp giữa hai mô hình (Bảng 6.4): filet mignon ↔ steak (66 ảnh với ResNet-50, 76 với DeiT-S), prime rib ↔ steak, chocolate cake ↔ chocolate mousse, beef tartare ↔ tuna tartare. Số trong ngoặc là số ảnh test bị nhầm theo cả hai chiều (seed 0, full fine-tuning). Xét từng ảnh test (seed 0, full fine-tuning), cả hai mô hình cùng đúng ở 20 810 ảnh (82,4%), cùng sai ở 2 152 ảnh (8,5%), chỉ ResNet-50 sai ở 1 380 ảnh và chỉ DeiT-S sai ở 908 ảnh. Hệ số Jaccard của hai tập lỗi là 0,48 và tương quan độ chính xác theo lớp giữa hai mô hình là 0,95; nếu chọn đúng mô hình cho từng ảnh (oracle) thì độ chính xác là 91,48%.

Bảng 6.4: Các cặp lớp bị nhầm nhiều nhất trên test (full fine-tuning, seed 0). Cột "Chung" là các cặp ResNet-50 mà DeiT-S cũng nhầm, đếm theo ResNet-50.

ResNet-50	DeiT-S	Chung cho cả hai
filet mignon ↔ steak (66)	filet mignon ↔ steak (76)	filet mignon ↔ steak (66)
prime rib ↔ steak (47)	chocolate cake ↔ chocolate mousse (38)	prime rib ↔ steak (47)
chocolate cake ↔ chocolate mousse (47)	prime rib ↔ steak (36)	chocolate cake ↔ chocolate mousse (47)
beef tartare ↔ tuna tartare (39)	pork chop ↔ steak (33)	beef tartare ↔ tuna tartare (39)
pork chop ↔ steak (31)	beef tartare ↔ tuna tartare (32)	pork chop ↔ steak (31)
grilled salmon ↔ pork chop (27)	apple pie ↔ bread pudding (29)	grilled salmon ↔ pork chop (27)
filet mignon ↔ pork chop (25)	filet mignon ↔ pork chop (25)	filet mignon ↔ pork chop (25)
chocolate mousse ↔ tiramisu (22)	grilled salmon ↔ pork chop (22)	chocolate mousse ↔ tiramisu (22)

Bảng 6.5: Các lớp có độ chính xác thấp nhất (%, test, seed 0).

Lớp	ResNet-50	DeiT-S
steak	55,6	60,4
pork chop	66,0	64,4
huevos rancheros	64,0	72,8
apple pie	66,0	72,8
filet mignon	72,4	68,4
foie gras	67,6	75,2



Hình 6.3: Độ chính xác theo lớp: ResNet-50 so với DeiT-S, và các lớp khó nhất.

6.4 Chi phí tính toán

6.4.1 Chi phí huấn luyện

Bảng 6.6: Chi phí huấn luyện. Cột "chỉ GPU" dùng để so sánh (batch tổng hợp trên GPU); cột "thực tế" là thời gian mỗi epoch trong lần huấn luyện thật trên Kaggle (hai tiến trình chạy song song, mỗi tiến trình một GPU), bị giới hạn bởi bộ nạp dữ liệu (xem Mục 5.8). Peak mem là bộ nhớ đỉnh khi huấn luyện trong phép đo kiểm soát (batch 64).

Kiến trúc	Chế độ	Trainable (M / %)	Epoch	s/epoch (chỉ GPU)	phút/epoch (thực tế)	Tổng (giờ)	Peak mem (MB)
ResNet-50	From-scratch	23,71 / 100,0%	40	192,9	4,7	3,15	3 161
ResNet-50	Linear probing	0,21 / 0,9%	10	70,5	4,5	0,78	470
ResNet-50	Freeze một phần	15,17 / 64,0%	10	86,4	4,9	0,79	646
ResNet-50	Full fine-tuning	23,71 / 100,0%	10	195,0	5,2	0,82	3 161
DeiT-S	From-scratch	21,70 / 100,0%	40	191,0	5,1	3,40	2 502
DeiT-S	Linear probing	0,04 / 0,2%	10	58,3	4,9	0,79	287
DeiT-S	Freeze một phần	10,69 / 49,2%	10	122,8	4,7	0,79	1 336
DeiT-S	Full fine-tuning	21,70 / 100,0%	10	194,4	4,8	0,83	2 502

Theo số đo chỉ GPU, linear probing nhanh hơn full fine-tuning khoảng 2,8 lần ở ResNet-50 (70,5 so với 195,0 s/epoch) và 3,3 lần ở DeiT-S (58,3 so với 194,4 s/epoch). Freeze một phần tiết kiệm nhiều ở ResNet-50 (chỉ còn 86,4 s/epoch, bộ nhớ huấn luyện còn 20% so với full fine-tuning) nhưng ít hơn ở DeiT-S (122,8 s/epoch, bộ nhớ còn 53%; Bảng 4.1), vì nửa sau các khối Transformer phải backward. Thời gian thực tế mỗi epoch lại gần như không đổi (4,5–5,2 phút) giữa các chế độ: bộ nạp dữ liệu, không phải GPU, là nút cổ chai.

6.4.2 Chi phí suy luận

Bảng 6.7: Chi phí suy luận (chỉ phụ thuộc kiến trúc; AMP fp16; Tesla T4, PyTorch 2.11.0+cu128, timm 1.0.30, đo bằng `cost_bench.py`). Latency batch 1 bị chi phối bởi chi phí khởi chạy kernel nên không tỷ lệ với GMACs.

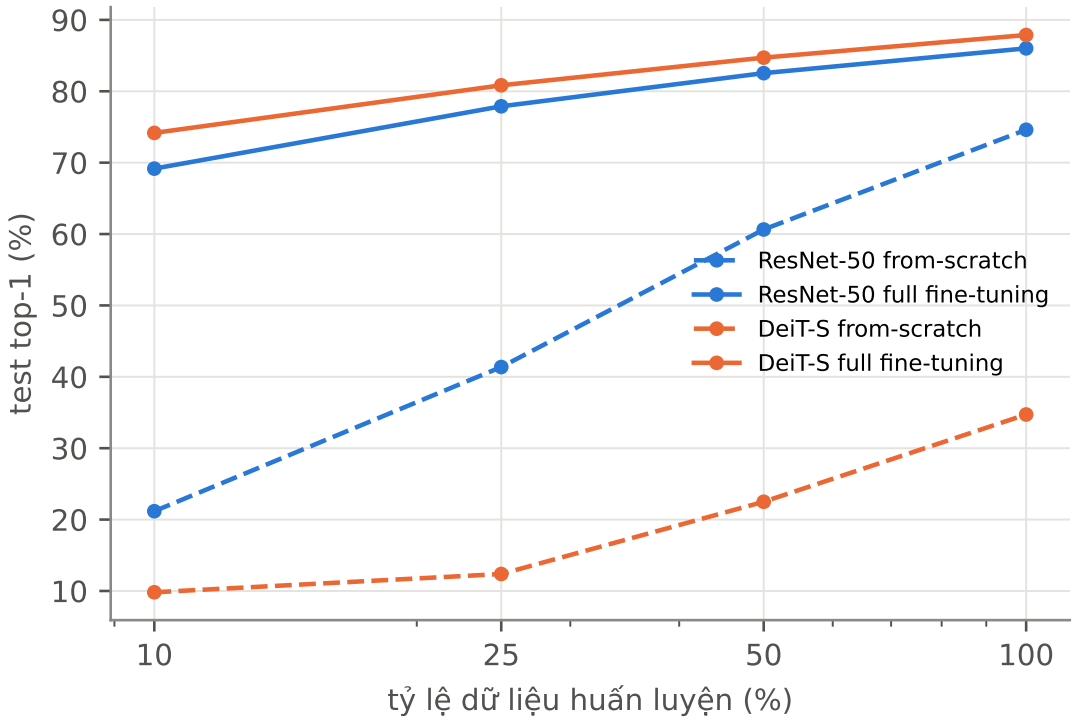
Kiến trúc	Tham số (M)	GMACs	img/s (batch 128)	Latency batch 1 (ms): median / p95	Peak mem (MB): batch 1 / 128
ResNet-50	23,71	4,110	1031,8	7,23 / 8,42	117 / 822
DeiT-S	21,70	4,608	1156,0	7,09 / 8,53	106 / 472
ConvNeXt-T (mở rộng)	27,90	4,470	678,7	8,09 / 9,31	136 / 1 231
Swin-T (mở rộng)	27,60	4,509	452,1	13,23 / 15,07	140 / 1 719

ResNet-50 và DeiT-S gần nhau về latency batch 1 và thông lượng (GMACs chênh khoảng 12%) (1 032 so với 1 156 ảnh/s). Hai kiến trúc mở rộng chậm hơn rõ rệt, nhất là Swin-T (khoảng 452 ảnh/s, latency 13,2 ms), dù số GMACs thấp hơn DeiT-S khoảng 2%.

6.5 Ablation theo kích thước dữ liệu

Bảng 6.8: Top-1 test (%) theo tỷ lệ dữ liệu huấn luyện; cột cuối là mức tăng từ 10% lên 100% dữ liệu (điểm phần trăm). Một seed.

Kiến trúc	Chế độ	10%	25%	50%	100%	Tăng
ResNet-50	From-scratch	21,17	41,37	60,64	74,61	+53,4
ResNet-50	Full fine-tuning	69,16	77,89	82,54	86,01	+16,8
DeiT-S	From-scratch	9,82	12,38	22,50	34,72	+24,9
DeiT-S	Full fine-tuning	74,17	80,84	84,71	87,88	+13,7



Hình 6.4: Độ chính xác theo tỷ lệ dữ liệu huấn luyện.

Với 10% dữ liệu (70 ảnh mỗi lớp), full fine-tuning đã đạt 69,16% (ResNet-50) và 74,17% (DeiT-S), trong khi from-scratch chỉ đạt 21,17% và 9,82%. Khoảng cách giữa pretrained và from-scratch thu hẹp dần khi dữ liệu tăng: ở ResNet-50 từ 48 xuống 11,4 điểm, ở DeiT-S từ 64,3 xuống 53,2 điểm.

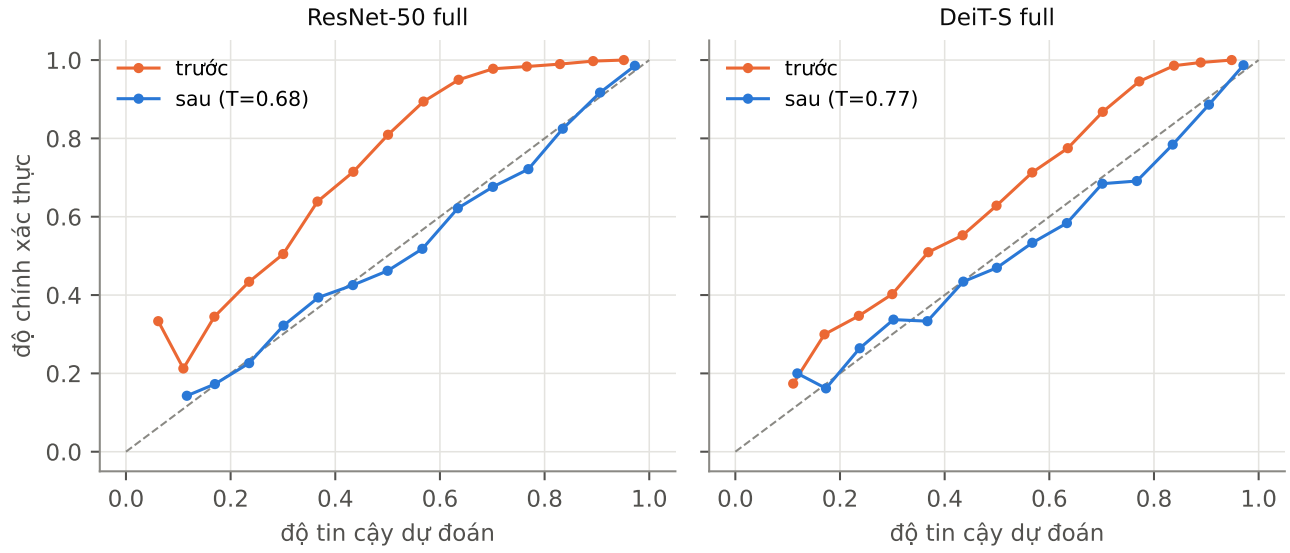
6.6 Kết quả phân tích bổ sung (ngoài yêu cầu tối thiểu)

6.6.1 Hiệu chỉnh độ tin cậy

Bảng 6.9: Hiệu chỉnh trên test, trước và sau temperature scaling (nhiệt độ T khớp trên logits validation). ECE tính theo phần trăm, 15 bin.

Kiến trúc	Chế độ	ECE trước	ECE sau	NLL trước	NLL sau	Brier trước	Brier sau	T
DeiT-S	Full fine-tuning	13,55	2,19	0,579	0,464	0,197	0,178	0,77
DeiT-S	Linear probing	21,99	4,08	1,319	1,108	0,469	0,407	0,68
DeiT-S	Freeze một phần	11,75	2,46	0,538	0,443	0,184	0,169	0,80
DeiT-S	From-scratch	13,62	1,51	2,798	2,649	0,814	0,786	0,72
ResNet-50	Full fine-tuning	23,93	1,63	0,777	0,520	0,266	0,200	0,68
ResNet-50	Linear probing	27,38	2,69	1,487	1,148	0,517	0,420	0,58
ResNet-50	Freeze một phần	23,80	1,57	0,848	0,594	0,293	0,226	0,68
ResNet-50	From-scratch	22,87	1,39	1,193	0,943	0,417	0,349	0,67

Tất cả mô hình đều *thiếu tự tin* (độ tin cậy thấp hơn độ chính xác thực) và nhiệt độ khớp được đều nhỏ hơn 1 (0,58–0,80), tức cần làm các xác suất nhọn hơn, ngược với hiện tượng quá tự tin thường thấy ở mạng sâu. Full fine-tuning có ECE 23,9% (ResNet-50) và 13,5% (DeiT-S) trước hiệu chỉnh, giảm xuống 1,6% và 2,2% sau temperature scaling. Hình 6.5 cho thấy đường reliability nằm trên đường chéo trước hiệu chỉnh.



Hình 6.5: Biểu đồ reliability trước và sau temperature scaling.

6.6.2 Ensemble

Bảng 6.10: Top-1 test (%) của ensemble trung bình xác suất.

Cấu hình	Số mô hình	Top-1 (%)
ResNet-50 (đơn, seed 0)	1	86,01
DeiT-S (đơn, seed 0)	1	87,88
ResNet-50 ensemble 3 seed	3	87,23
DeiT-S ensemble 3 seed	3	88,92
ResNet-50 + DeiT-S (seed 0)	2	88,91
ResNet-50 + DeiT-S (3 + 3 seed)	6	89,57

Ensemble 3 seed cùng kiến trúc tăng 1,2 điểm (ResNet-50) và 1,0 điểm (DeiT-S) so với mô hình đơn seed 0. Ensemble một ResNet-50 và một DeiT-S đã đạt 88,91%, hơn mô hình đơn tốt nhất 1,03 điểm, và ensemble $3 + 3$ đạt 89,57%. Mức tăng của ensemble khác họ đến từ việc hai mô hình sai ở các ảnh khác nhau một phần (Jaccard của hai tập lỗi 0,48); nhóm không tách riêng ảnh hưởng của việc DeiT-S mạnh hơn ResNet-50 (Mục 7.4).

6.6.3 Độ bền vững trước nhiễu

Bảng 6.11 cho tỷ lệ độ chính xác còn lại (so với độ chính xác sạch trên cùng tập con) ở mức nhiễu mạnh nhất (mức 3) của từng loại; chi tiết theo cả ba mức nằm ở Phụ lục B. Tập con gồm 50 ảnh mỗi lớp (5 050 ảnh), nhiễu được sinh xác định từ seed cố định; Bảng 6.11 là seed 0, còn Bảng 6.12 lặp lại phép đo trên cả ba seed.

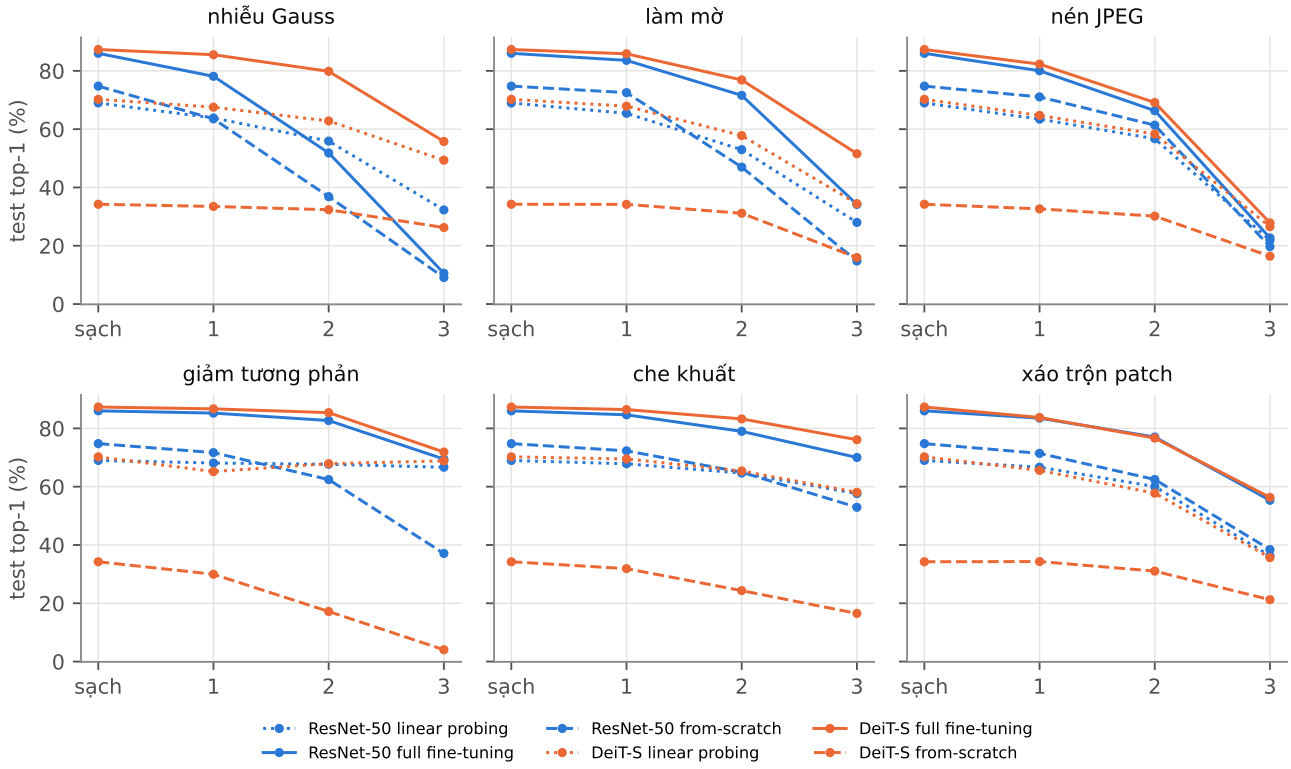
Bảng 6.11: Độ chính xác còn lại (%) ở mức nhiễu 3 so với ảnh sạch. Cột "Sạch" là top-1 (%) trên tập con.

Kiến trúc	Chế độ	Sạch	Nhiễu Gauss	Mờ	JPEG	Tương phản	Che khuất	Xáo trộn mảnh	TB
ResNet-50	From-scratch	74,77	12,2	19,8	26,4	49,6	70,8	51,4	69,1
ResNet-50	Linear probing	68,99	46,8	40,6	31,8	96,6	83,5	52,6	80,2
ResNet-50	Freeze một phần	83,92	51,5	46,0	33,4	96,5	86,9	52,4	81,8
ResNet-50	Full fine-tuning	86,02	12,3	39,7	26,4	80,8	81,4	64,3	76,6
DeiT-S	From-scratch	34,24	76,7	46,6	47,9	11,9	48,2	62,0	75,1
DeiT-S	Linear probing	70,24	70,3	49,1	37,8	98,0	82,7	50,8	82,5
DeiT-S	Freeze một phần	88,34	84,8	58,6	45,7	98,7	89,4	65,5	88,1
DeiT-S	Full fine-tuning	87,33	63,9	59,1	31,9	82,3	87,2	64,5	84,1

Bảng 6.12: Độ chính xác còn lại (%) ở mức nhiễu 3, mean ± std trên 3 seed.

Kiến trúc	Chế độ	Số seed	Nhiễu Gauss	Mờ	JPEG	Tương phản	Che khuất	Xáo trộn mảnh
ResNet-50	Linear probing	3	47,4 ± 0,6	39,4 ± 1,1	32,5 ± 0,7	96,7 ± 0,2	83,8 ± 0,3	52,5 ± 0,2
ResNet-50	Freeze một phần	3	51,8 ± 0,3	46,3 ± 0,3	33,5 ± 0,4	96,6 ± 0,2	86,4 ± 0,9	52,3 ± 0,1
ResNet-50	Full fine-tuning	3	12,5 ± 5,9	40,9 ± 1,6	24,2 ± 1,9	83,2 ± 2,1	82,5 ± 2,1	63,0 ± 1,2
DeiT-S	Linear probing	3	70,0 ± 0,5	49,0 ± 0,6	38,3 ± 0,6	97,4 ± 0,6	82,6 ± 0,4	50,5 ± 0,4
DeiT-S	Freeze một phần	3	84,8 ± 0,4	58,0 ± 0,6	45,7 ± 0,1	98,6 ± 0,2	89,4 ± 0,1	65,9 ± 0,4
DeiT-S	Full fine-tuning	3	60,1 ± 3,3	56,3 ± 2,5	31,4 ± 0,8	84,8 ± 2,2	88,0 ± 0,8	64,3 ± 0,3

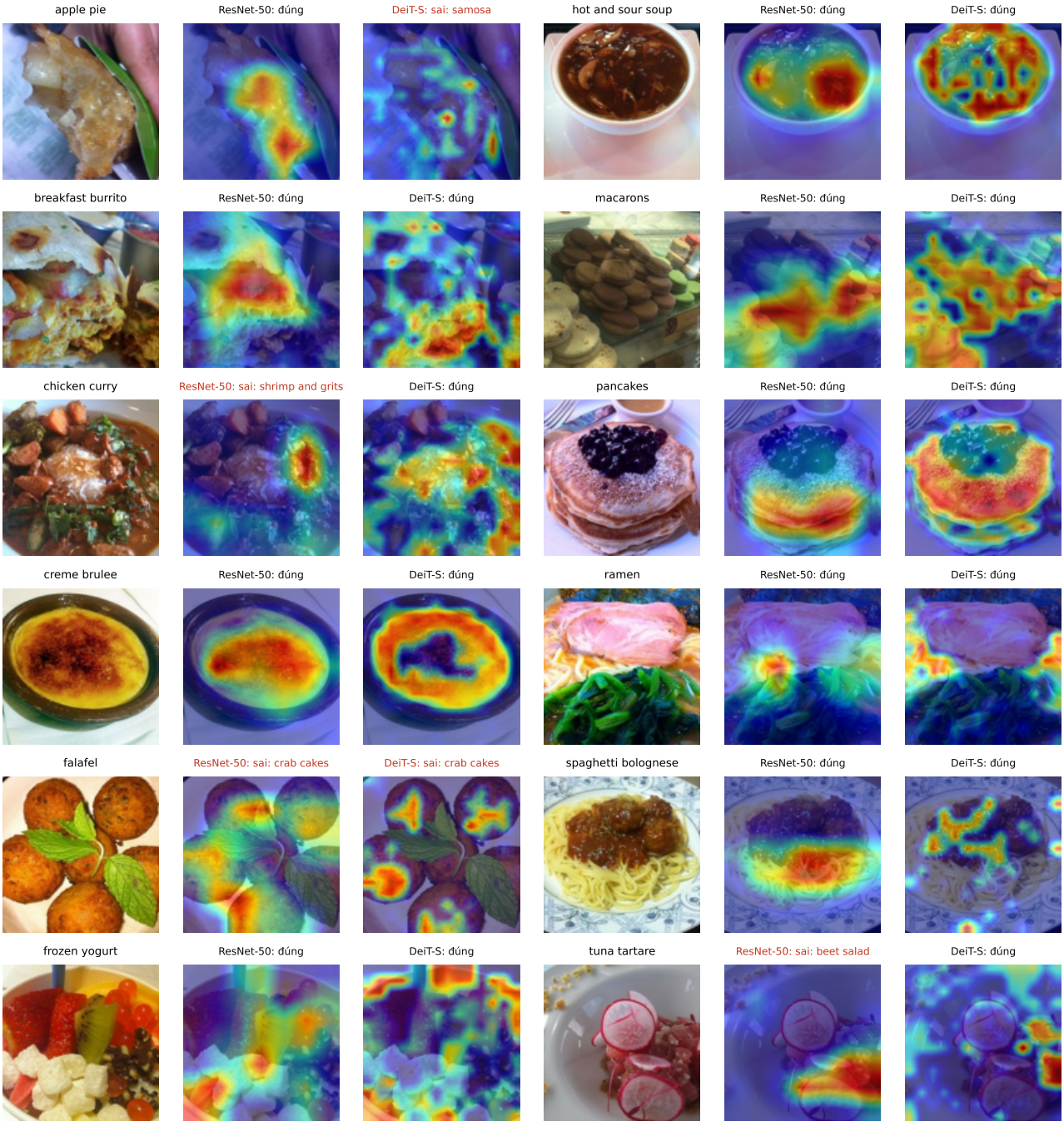
Ở ResNet-50, full fine-tuning kém bền hơn freeze một phần với nhiễu Gauss, JPEG, mờ và giảm tương phản, nhưng *bền hơn* với xáo trộn mảnh ($63{,}0 \pm 1{,}2\%$ so với $52{,}3 \pm 0{,}1\%$); ở DeiT-S hai chế độ ngang nhau với xáo trộn mảnh (64,3 so với 65,9%).



Hình 6.6: Độ chính xác theo mức nhiễu của từng loại.

Full fine-tuning của ResNet-50 là cấu hình pretrained mong manh nhất trước nhiễu Gauss: chỉ còn 12,3% độ chính xác sạch ở mức 3 (seed 0), so với 46,8% (linear probing) và 51,5% (freeze một phần). Trên ba seed (Bảng 6.12) kết luận giữ nguyên nhưng có độ biến thiên lớn ở full fine-tuning: $12{,}5 \pm 5{,}9\%$ với ResNet-50 và $60{,}1 \pm 3{,}3\%$ với DeiT-S, so với $51{,}8 \pm 0{,}3\%$ và $84{,}8 \pm 0{,}4\%$ ở freeze một phần. DeiT-S giữ được nhiều hơn ở nhiễu Gauss (63,9 đến 84,8% còn lại với các chế độ pretrained), trong khi cả hai đều suy giảm mạnh với JPEG chất lượng thấp và làm mờ; che khuất và giảm tương phản gây ít thiệt hại nhất với các mô hình pretrained.

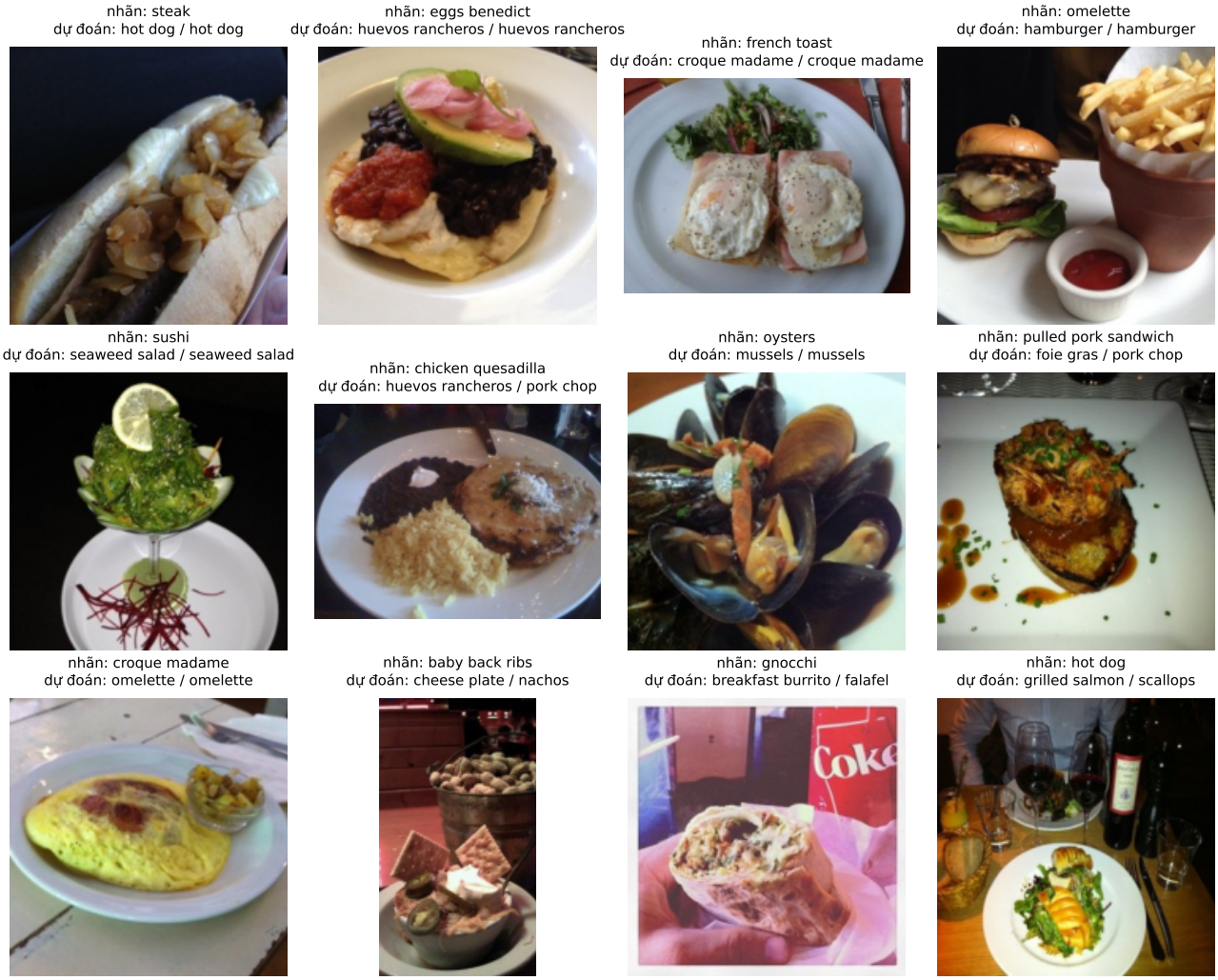
6.6.4 Diễn giải: Grad-CAM và ảnh huấn luyện loss cao



Hình 6.7: Grad-CAM của ResNet-50 và DeiT-S (full fine-tuning, seed 0) trên các ảnh test được chọn trước (một ảnh đầu tiên của mỗi lớp trong tập con xác định); đây là diễn giải định tính.

Trong 200 ảnh huấn luyện có loss cao nhất của mỗi mô hình, 74 ảnh trùng nhau giữa hai mô hình và 47 ảnh được cả hai dự đoán cùng một lớp khác với nhãn. Độ chính xác trên chính tập huấn luyện chỉ là 88,8% (ResNet-50) và 91,3% (DeiT-S), phù hợp với việc nhãn huấn luyện của

Food-101 có nhiễu chủ ý. Hình 6.8 minh họa các ví dụ này.



Hình 6.8: Ảnh huấn luyện có loss cao nhất và nhãn gốc/dự đoán của hai mô hình.

6.7 Thí nghiệm bổ sung sau giao thức

Các thí nghiệm ở Mục 5.6 đều là một seed và được báo cáo bên cạnh kết quả giao thức. Chi tiết quét LR nằm ở Phụ lục B.

6.7.1 LR của freeze một phần và LR ở biên

Trong lượt quét ngắn (3 epoch), mức LR mới cao hơn LR đã chọn ở bốn trong năm nhóm được kiểm tra; ngoại lệ là DeiT-S linear probing, nơi LR đã chọn ($9 \cdot 10^{-3}$) hóa ra là điểm tối ưu bên trong lưới. Bốn lần chạy 10 epoch ở các LR mới cho kết quả khác nhau (Bảng 6.13):

- ResNet-50 linear probing ở $2{,}7 \cdot 10^{-2}$: validation 64,12% so với 64,44% của giao thức, test 68,97% so với 69,26%. Lợi thế +1,0 điểm của lượt quét ngắn biến mất.
- ResNet-50 full fine-tuning ở $2{,}7 \cdot 10^{-3}$: validation 80,38% so với 80,87%, test 84,99% so với 86,01%, tức *thấp hơn*. Lượt quét ngắn (+2,3 điểm) đã thiên về LR lớn như dự đoán ở phần hạn chế.
- ResNet-50 freeze một phần ở $2{,}7 \cdot 10^{-3}$: validation $81{,}07 \pm 0{,}27\%$ và test $85{,}69 \pm 0{,}16\%$ trên năm seed (85,94, 85,70, 85,71, 85,59, 85,50), so với $84{,}09 \pm 0{,}08\%$ của giao thức. Cải thiện +1,6 điểm lớn hơn nhiều lần độ lệch chuẩn. Các seed 1–4 được chạy thêm vì kết quả này làm thay đổi kết luận về RQ1 (quy tắc ở Mục 5.6).
- DeiT-S freeze một phần ở 10^{-3}: validation 83,50% so với 83,78%, test 88,59% so với 88,36%; khác biệt nằm trong khoảng nhiễu giữa seed.

Hệ quả quan trọng nhất là ở ResNet-50: khi freeze một phần có LR riêng, nó đạt $85{,}69 \pm 0{,}16\%$ (năm seed), còn full fine-tuning đạt $86{,}03 \pm 0{,}14\%$ (năm seed: ba seed của giao thức cộng hai seed chạy thêm ở cùng LR $9 \cdot 10^{-4}$). Khoảng cách thu hẹp từ 1,9 xuống 0,34 điểm nhưng không biến mất: kiểm định Welch cho $t = -3{,}47$ ($p \approx 0{,}009$, chưa hiệu chỉnh đa so sánh), tức một thiếu hụt nhỏ nhưng nhất quán của freeze một phần. Lưu ý $2{,}7 \cdot 10^{-3}$ là mức LR cao nhất đã thử cho chế độ này nên LR tối ưu của freeze một phần có thể còn cao hơn, và full fine-tuning ở LR $2{,}7 \cdot 10^{-3}$ thấp hơn (84,99%, seed 0). Ở DeiT-S, LR riêng không đổi kết quả đáng kể, nên việc freeze một phần vượt full fine-tuning ở DeiT-S không do dùng chung LR.

Bảng 6.13: Các lần chạy bổ sung 10 epoch so với lần chạy giao thức cùng mô hình (seed 0, trừ khi ghi rõ). ECE tính theo phần trăm.

Kiến trúc	Thiết lập	Top-1 test (%)	ECE (%)	NLL
ResNet-50	Giao thức (Mixup/CutMix, label smoothing 0,1)	86,01	23,93	0,777
ResNet-50	Full fine-tuning, LR $2.7 \cdot 10^{-3}$ (độ nhạy, seed 0)	84,99	23,45	0,803
ResNet-50	Full fine-tuning, LR $9 \cdot 10^{-4}$ (LR giao thức, thêm seed, seed 3)	86,26	22,98	0,759
ResNet-50	Full fine-tuning, LR $9 \cdot 10^{-4}$ (LR giao thức, thêm seed, seed 4)	85,91	27,16	0,833
ResNet-50	Linear probing, LR $2.7 \cdot 10^{-2}$ (độ nhạy, seed 0)	68,97	27,10	1,495
ResNet-50	Freeze một phần, LR $2.7 \cdot 10^{-3}$ (độ nhạy, seed 0)	85,94	23,17	0,771
ResNet-50	Freeze một phần, LR $2.7 \cdot 10^{-3}$ (độ nhạy, seed 1)	85,70	24,69	0,802
ResNet-50	Freeze một phần, LR $2.7 \cdot 10^{-3}$ (độ nhạy, seed 2)	85,71	24,95	0,806
ResNet-50	Freeze một phần, LR $2.7 \cdot 10^{-3}$ (độ nhạy, seed 3)	85,59	22,36	0,767
ResNet-50	Freeze một phần, LR $2.7 \cdot 10^{-3}$ (độ nhạy, seed 4)	85,50	25,94	0,828
ResNet-50	Không Mixup/CutMix, label smoothing 0,1	86,83	9,69	0,564
ResNet-50	Không Mixup/CutMix, không label smoothing	86,80	2,69	0,484
DeiT-S	Giao thức (Mixup/CutMix, label smoothing 0,1)	87,88	13,55	0,579
DeiT-S	Layer-wise LR decay 0,75	88,12	13,03	0,551
DeiT-S	Không Mixup/CutMix, label smoothing 0,1	87,17	8,16	0,568
DeiT-S	Không Mixup/CutMix, không label smoothing	86,96	2,14	0,475
DeiT-S	Freeze một phần, LR 10^{-3} (độ nhạy, seed 0)	88,59	11,81	0,533

6.7.2 Nguồn gốc của sự thiếu tự tin

Bỏ Mixup/CutMix (giữ label smoothing) làm ECE giảm từ 23,9% xuống 9,7% ở ResNet-50 và từ 13,5% xuống 8,2% ở DeiT-S; bỏ thêm label smoothing đưa ECE xuống 2,7% và 2,1% (NLL 0,777 → 0,564 → 0,484 và 0,579 → 0,568 → 0,475). Như vậy các mục tiêu mềm là nguyên nhân chính của sự thiếu tự tin, với Mixup/CutMix đóng góp phần lớn ở ResNet-50. Ảnh hưởng đến độ chính xác lại khác nhau giữa hai họ: bỏ Mixup/CutMix *tăng* ResNet-50 lên 86,83% (+0,8 điểm so với giao thức, seed 0) và *giảm* DeiT-S xuống 87,17% (−0,7 điểm).

6.7.3 Layer-wise LR decay

Với hệ số 0,75, DeiT-S full fine-tuning đạt 88,12% so với 87,88% (seed 0; +0,24 điểm, cùng cỡ với độ lệch chuẩn giữa seed) và vẫn thấp hơn freeze một phần (88,36% seed 0); ECE 13,0% không đổi. Đây là bằng chứng yếu theo hướng ủng hộ giả thuyết rằng các block thấp bị cập nhật quá mức khi fine-tune toàn bộ, nhưng không đủ để kết luận.

Chương 7

PHÂN TÍCH VÀ THẢO LUẬN

Các nhận định dưới đây dựa trên số liệu ở Chương 6. Với cấu hình 3 seed, độ lệch chuẩn là 0,07–0,24 điểm; mọi khác biệt được diễn giải đều lớn hơn mức này, còn các cấu hình một seed (from-scratch, mở rộng, ablation) chỉ được dùng cho nhận xét về xu hướng. Các hạn chế được nêu ở Mục 9.2.

7.1 CNN và Transformer

Kết luận về "họ nào tốt hơn" phụ thuộc vào chế độ huấn luyện, cặp kiến trúc và cả công thức augmentation, nên không có câu trả lời một chiều. Khi có pretraining, DeiT-S vượt ResNet-50 ở linear probing (70,12 so với 69,25%, chênh lệch lớn hơn độ lệch chuẩn của hai mô hình), ở freeze một phần (88,27 so với 84,09%) và ở full fine-tuning (87,73 so với 85,99%). Ba điều làm giảm sức nặng của kết luận này: (i) cặp thứ hai cho bức tranh khác: ConvNeXt-T và Swin-T cùng đạt khoảng 88,9% khi full fine-tuning, tức khác biệt giữa CNN và ViT biến mất; (ii) ở freeze một phần, phần lớn khoảng cách 4,2 điểm đến từ việc dùng chung LR của full fine-tuning: khi ResNet-50 có LR riêng, nó đạt $85{,}69 \pm 0{,}16\%$ (năm seed) và khoảng cách còn 2,9 điểm so với DeiT-S (88,59% với LR riêng; một seed); (iii) ở full fine-tuning, công thức Mixup/CutMix không trung tính giữa hai họ: bỏ nó làm ResNet-50 tăng lên 86,83% và DeiT-S giảm xuống 87,17%, nên khoảng cách 1,7 điểm của giao thức còn 0,3 điểm, không phân biệt được với nhiễu giữa seed (một seed mỗi thiết lập). Vì vậy phần lớn ưu thế của DeiT-S trong giao thức gắn với công thức huấn luyện chung và việc chọn LR, hơn là với họ kiến trúc. Điều này cũng phù hợp với việc ResNet-50 A1 và DeiT được pretrain bằng các công thức khác nhau (Chương 3). Ở from-scratch thì CNN rõ ràng thuận lợi hơn: ResNet-50 đạt 74,61% và ConvNeXt-T 76,82%, trong khi DeiT-S chỉ 34,72% và Swin-T 59,53% sau 40 epoch. Swin-T, vốn có thiên kiến cục bộ qua cửa sổ dịch, đứng giữa hai nhóm. Các lần chạy mở rộng chỉ có một seed và dùng LR chọn từ lượt quét ngắn, nên không đủ bằng chứng để xếp hạng ConvNeXt-T và Swin-T với nhau.

7.2 Vai trò của pretraining

RQ2. Pretraining là yếu tố quyết định nhất trong toàn bộ thực nghiệm: lợi ích lớn hơn nhiều so với khác biệt giữa các chế độ freeze hay giữa hai họ kiến trúc. Ở 100% dữ liệu, full fine-tuning hơn from-scratch 11,4 điểm với ResNet-50 và 53,2 điểm với DeiT-S. Ablation theo dữ liệu (Bảng 6.8) cho thấy lợi ích càng lớn khi dữ liệu càng ít: với 10% dữ liệu, khoảng cách là 48,0 điểm với ResNet-50 và 64,3 với DeiT-S, và chỉ thu hẹp dần khi tăng dữ liệu. Nghĩa là DeiT-S, với thiên kiến quy nạp yếu hơn, phụ thuộc vào dữ liệu hoặc pretrain nhiều hơn. Tuy vậy, đối chứng 100 epoch cho thấy một phần khoảng cách là do huấn luyện chưa đủ lâu: DeiT-S from-scratch tăng từ 34,72% (40 epoch) lên 67,52% (100 epoch), vẫn thấp hơn ResNet-50 sau 40 epoch (74,61%), và không kiểm tra được liệu thêm epoch có thu hẹp tiếp khoảng cách hay không. Vì vậy phát biểu "ViT cần pretrain" đúng ở lịch huấn luyện ngắn, còn ở lịch dài thì khoảng cách là một phần của sự đánh đổi giữa dữ liệu và thời gian huấn luyện. Cũng cần nhắc rằng lợi thế của pretrain có thể một phần đến từ sự gần gũi miền dữ liệu giữa ImageNet-1k và Food-101 (Mục 9.2).

7.3 Ảnh hưởng của fine-tuning và freeze

RQ1. Linear probing là mức chi phí thấp nhất, nhưng độ chính xác thấp hơn 16–18 điểm so với full fine-tuning; nó chỉ phù hợp khi tài nguyên cực kỳ hạn chế. Điểm đáng chú ý nằm ở giữa: freeze một phần cho độ chính xác gần với full fine-tuning với chi phí thấp hơn rõ rệt (Bảng 4.1). Ở ResNet-50, trong giao thức freeze layer4 thấp hơn full fine-tuning 1,9 điểm, nhưng phần lớn khoảng cách này là do LR: với LR riêng ($2{,}7 \cdot 10^{-3}$, năm seed) freeze một phần đạt $85{,}69 \pm 0{,}16\%$, thấp hơn full fine-tuning ($86{,}03 \pm 0{,}14\%$, năm seed) chỉ 0,34 điểm (Welch $p \approx 0{,}009$): một thiếu hụt nhỏ nhưng có ý nghĩa thống kê, trong khi chỉ tốn khoảng 44% thời gian GPU và khoảng 20% bộ nhớ huấn luyện. Kết luận "full fine-tuning tốt hơn freeze một phần ở ResNet-50" của giao thức vì thế chỉ đúng với LR dùng chung, và nên được hiểu là một thiên lệch của việc chọn LR chứ không phải tính chất của chế độ. (Lưu ý LR $2{,}7 \cdot 10^{-3}$ là mức cao nhất đã thử cho chế độ này, nên giá trị tối ưu có thể còn cao hơn.) Ở DeiT-S, freeze nửa sau các khối không những rẻ hơn (122,8 so với 194,4 s/epoch) mà còn *cao hơn* full fine-tuning 0,54 điểm, và kết quả này bền với việc đổi LR (LR riêng 10^{-3} cho 88,59% ở một seed). Hai giải thích đã được kiểm tra: dùng chung LR (không phải nguyên nhân) và các block thấp bị cập nhật quá mức. Với layer-wise LR decay 0,75, full fine-tuning tăng 0,24 điểm lên 88,12%, thu hẹp khoảng cách với freeze một phần (88,36%, cùng seed) nhưng không xóa hẳn và nằm trong khoảng nhiễu giữa seed; đây là bằng chứng yếu ủng hộ giả thuyết thứ hai, chưa phải kết luận. Cũng không loại trừ vai trò của Mixup/CutMix: bỏ nó làm DeiT-S full fine-tuning giảm, nên không phải nguyên nhân khiến full tụt hậu. Ablation cho thấy khi dữ liệu ít, full fine-tuning vẫn hiệu quả (69–74% ở 10% dữ liệu), nên với bài toán này không có dấu hiệu fine-tuning toàn bộ bị quá khớp trên dữ liệu nhỏ; nhóm không ablate riêng freeze một

phần theo dữ liệu nên không kết luận được về điểm này. Về độ bền vững, full fine-tuning là cấu hình pretrained kém bền nhất trước nhiễu Gauss ở cả hai kiến trúc và trên cả ba seed (Bảng 6.12): $12{,}5 \pm 5{,}9\%$ so với $51{,}8 \pm 0{,}3\%$ ở ResNet-50 và $60{,}1 \pm 3{,}3\%$ so với $84{,}8 \pm 0{,}4\%$ ở DeiT-S. Độ chính xác sạch cao hơn không đồng nghĩa với độ bền cao hơn. Ngoại lệ là xáo trộn mảnh ở ResNet-50, nơi full fine-tuning bền hơn.

7.4 Phân tích lỗi

7.4.1 Hiệu năng theo lớp

Các lớp khó nhất là các món có hình thức tương tự với món khác: steak, pork chop, huevos rancheros, apple pie, filet mignon và foie gras đều có độ chính xác dưới khoảng 76% ở cả hai mô hình (Bảng 6.5). Khó khăn nằm ở cùng một số lớp ở cả CNN và ViT, tức nguyên nhân chủ yếu thuộc về dữ liệu (định nghĩa lớp mơ hồ hoặc nhãn nhiễu), không phải đặc thù của một họ kiến trúc.

7.4.2 Các mẫu dự đoán sai

Các cặp nhầm nhiều nhất (Bảng 6.4) là những món thực sự gần nhau (filet mignon/steak, chocolate cake/chocolate mousse, beef tartare/tuna tartare). Một phần khác của lỗi đến từ nhãn: độ chính xác trên chính tập huấn luyện chỉ 88,8% (ResNet-50) và 91,3% (DeiT-S) dù mô hình đã thấy các ảnh này, và trong 200 ảnh huấn luyện có loss cao nhất, 74 ảnh trùng giữa hai mô hình và 47 ảnh được cả hai dự đoán cùng một lớp khác nhãn (Hình 6.8); đó là dấu hiệu nhãn sai chứ không phải lỗi ngẫu nhiên của một mô hình. Điều này cũng giải thích vì sao validation (lấy từ phần train nhiễu) thấp hơn test (sạch): ví dụ ResNet-50 full có 80,87% validation so với 86,01% test. Hệ quả là con số validation chỉ nên dùng để chọn siêu tham số, không dùng để so sánh với test.

7.4.3 Kiểu lỗi của CNN và Transformer

Các cặp nhầm hàng đầu gần như trùng nhau giữa ResNet-50 và DeiT-S (Bảng 6.4) và độ chính xác theo lớp của hai mô hình tương quan rất cao (0,95), nên phần lớn lỗi có tính hệ thống theo dữ liệu hơn là theo kiến trúc. Tuy vậy, hai mô hình không sai ở cùng các ảnh: chỉ 2 152 trong 4 440 ảnh mà ít nhất một mô hình sai là lỗi chung (Jaccard 0,48), và $1\,380 + 908$ ảnh chỉ một mô hình sai. Điều này giải thích vì sao ensemble ResNet-50 + DeiT-S (88,91%) hơn mô hình tốt nhất 1,03 điểm, tương đương mức tăng của ensemble ba seed cùng kiến trúc (+1,0 đến +1,2 điểm); ensemble $3 + 3$ (89,57%) cao hơn cả hai, nhưng nhóm không tách được đa dạng giữa hai họ khỏi đa dạng giữa các seed. Ngưỡng oracle 91,48% cho thấy còn khoảng 2,6 điểm có thể khai thác nếu

biết nên tin mô hình nào. **RQ4.** Tóm lại, hai họ mắc lỗi giống nhau ở phần lớn các lớp khó nhưng khác nhau ở một phần đáng kể các ảnh riêng lẻ, nên lợi ích của việc kết hợp chúng là khiêm tốn nhưng có thật.

7.5 Hiệu chỉnh, ensemble, độ bền vững và diễn giải

Mọi mô hình đều thiếu tự tin chứ không quá tự tin (Bảng 6.9, Hình 6.5), điều ngược với kỳ vọng thông thường. Các lần chạy bổ sung xác định nguyên nhân: bỏ Mixup/CutMix làm ECE giảm từ 23,9% xuống 9,7% (ResNet-50) và từ 13,5% xuống 8,2% (DeiT-S), bỏ thêm label smoothing đưa ECE về khoảng 2–3% (Bảng 6.13). Sự thiếu tự tin do vậy chủ yếu đến từ các mục tiêu mềm trong huấn luyện, nhất là Mixup/CutMix ở ResNet-50. Cái giá phải trả cho độ chính xác phụ thuộc mô hình: ResNet-50 không mất mà tăng (+0,8 điểm), DeiT-S giảm 0,7–0,9 điểm (một seed mỗi thiết lập). Temperature scaling với một tham số, khớp trên validation, đã đủ giảm ECE xuống 1,6–2,2% ở các mô hình full fine-tuning mà không đổi độ chính xác; vì validation nhiễu nhãn hơn test nên cần hiểu đây là hiệu chỉnh trên phân phối hơi khác. Về độ bền vững, full fine-tuning kém bền với nhiễu Gauss, JPEG và giảm tương phản ở cả hai kiến trúc (Mục 7.3); nhóm không kiểm tra nguyên nhân của hiện tượng này. Các bản đồ Grad-CAM (Hình 6.7) chỉ là diễn giải định tính trên một số ảnh được chọn trước; chúng không chứng minh quan hệ nhân quả.

7.6 Đánh đổi giữa hiệu năng và chi phí

RQ3. Trên đường Pareto chỉ-GPU (Hình 6.2), freeze một phần của DeiT-S là điểm đánh đổi tốt nhất: độ chính xác cao nhất trong các cấu hình pretrained của hai mô hình chính (88,27%) với 122,8 s/epoch và 1 336 MB. Với ResNet-50, freeze một phần (84,09% ở LR của giao thức, $85{,}69 \pm 0{,}16\%$ ở LR riêng, năm seed; 86,4 s/epoch) là điểm đánh đổi tốt, và với LR riêng chỉ mất khoảng 0,3 điểm so với full fine-tuning, còn linear probing chỉ có lợi khi thời gian là ràng buộc cứng (rẻ nhất nhưng thấp hơn 16–18 điểm). Full fine-tuning của ResNet-50 không nằm trên đường Pareto vì bị freeze một phần của DeiT-S vượt trội ở cả hai trục. Ở suy luận, ResNet-50 và DeiT-S gần như tương đương (Bảng 6.7): chênh lệch GMACs là khoảng 12% (đã đếm cả attention, Chương 3) nhưng độ trễ batch 1 chỉ lệch 0,14 ms, còn Swin-T chậm hơn rõ rệt dù GMACs thấp hơn DeiT-S khoảng 2%, cho thấy GMACs là chỉ số chưa đầy đủ. Về thời gian thực, trên Kaggle thời gian mỗi epoch gần như không đổi giữa các chế độ freeze (4,5–5,2 phút) vì bộ nạp dữ liệu là nút cổ chai, trong khi số đo chỉ GPU cho thấy khác biệt thật của chế độ huấn luyện; lợi ích của freeze về thời gian vì thế chỉ hiện thực hóa khi dữ liệu được cấp đủ nhanh, còn lợi ích về bộ nhớ thì luôn có.

Chương 8

TÁI LẬP THÍ NGHIỆM

Nhóm tách bạch **quy trình tái lập chuẩn** (chỉ cần mã nguồn và báo cáo) với **cách nhóm thực sự chạy** (các tối ưu để tiết kiệm thời gian). Nguyên tắc: tối ưu chỉ thay đổi *kết quả của một bước đến từ đâu*, không thay đổi *bước đó tính gì*.

Bảng 8.1: Quy trình chuẩn và cách nhóm thực sự chạy.

Bước	Quy trình chuẩn (chỉ repo)	Cách nhóm thực sự chạy
Dữ liệu	Tải Food-101 bằng torchvision, resize cạnh ngắn 256 bằng script của repo, chia theo `split.json` đã commit (kiểm tra SHA-256 và số ảnh khi khởi động)	Bản đã resize được đóng gói thành một dataset riêng trên Kaggle (không phân phối công khai) và sao chép vào đĩa tạm của phiên trước khi chạy; cùng mã, cùng kết quả
Huấn luyện	Mỗi lần chạy một tệp cấu hình YAML đã commit, chạy bằng lệnh CLI; có chế độ smoke test nhỏ (dưới 10 phút trên một GPU bất kỳ)	Hàng đợi chạy trên Kaggle với mỗi GPU một tiến trình độc lập; trạng thái, khóa lần chạy và checkpoint nằm trong một kho Hugging Face dùng chung, tự động tiếp tục khi mất phiên (đã kiểm tra khôi phục RNG cho kết quả trùng khớp từng bit trên mô hình thử); augmentation dùng seed theo từng mẫu nên không phụ thuộc số worker
Trọng số pretrained	timm tải theo tên và tag cố định	Cache tải sẵn, dùng chung
Kết quả	`results/*.json` (số liệu, đường cong, cấu hình, môi trường) trong repo; bảng và hình sinh lại từ JSON bằng notebook; trọng số cuối và logits đăng trên GitHub Release kèm checksum, có lệnh đánh giá lại để tái hiện số liệu test trong vài phút	Checkpoint và log trung gian nằm trên kho Hugging Face của nhóm (không phải sản phẩm nộp)

- **Môi trường:** đã kiểm tra với Python 3.13, PyTorch 2.11.0, torchvision 0.26.0, timm 1.0.30 trên Tesla T4. Tệp yêu cầu được tách theo bài (`requirements/base.txt`, `requirements/a1.txt`); `a1.txt` ghim `timm==1.0.30`, còn PyTorch và torchvision không được ghim vì Kaggle và Colab dùng bản dựng CUDA có sẵn (cài đè sẽ tải lại hàng GB và có thể làm hỏng runtime); các phiên bản đã kiểm tra được ghi trong README. Cấu hình khoa học (mô hình, chế độ, epoch, LR, seed, augmentation) nằm trong YAML được commit; đường dẫn và tham số máy (thư mục dữ liệu, số worker) đi qua biến môi trường hoặc tham số CLI.
- **Chi phí tái lập đầy đủ:** toàn bộ 88 lần chạy của giao thức tốn khoảng 52 GPU-giờ T4 (khoảng 64 giờ cùng 25 lần chạy bổ sung); phần tái lập trong notebook chỉ chạy một tập con. Sai khác giữa các lần chạy được ước lượng từ 3 seed và nêu trong báo cáo; kết quả không bảo đảm trùng từng bit giữa các loại GPU khác nhau.

- **Cấu trúc mã:** mã của A1 nằm trong thư mục `A1/`; các tiện ích dùng chung giữa các bài (seed và ghi môi trường, lưu checkpoint nguyên tử, ghi kết quả JSON, đo chi phí) nằm trong package `common/` cài được từ thư mục gốc repo (`pip install -e .`). Thí nghiệm là notebook mỏng gọi vào package và các script `python -m`.
- **Kiểm tra "fresh clone"** trước khi nộp A1.2: mở notebook `A1/notebooks/A1.ipynb` trên một runtime Colab mới, không có Drive hay dữ liệu cache. Notebook có ba chế độ: `load` sinh lại các bảng và hình từ `results/*.json` và logits trong Release rồi so khớp với bản đã commit; `evaluate` đánh giá lại trọng số đã phát hành trên tập test; `rerun` huấn luyện lại một tập con nhỏ (mặc định một lần chạy linear probing) để so sánh với kết quả đã công bố.
- **Phát hành:** trọng số cuối và logits đăng trên GitHub Release kèm `SHA256SUMS`. Nhóm không đăng dữ liệu gốc, bản cache đã resize, hay ảnh đã làm nhiễu.

Chương 9

KẾT LUẬN

9.1 Tổng kết kết quả

Thực nghiệm gồm 88 lần chạy trên Food-101 (70 700 ảnh train, 5 050 validation, 25 250 test) và cho các câu trả lời sau, kèm mức tin cậy.

- **RQ1 – fine-tuning và freeze.** Freeze một phần cho độ chính xác gần với full fine-tuning với chi phí thấp hơn: ở ResNet-50 thấp hơn 1,9 điểm ở LR của giao thức (84,09% so với 85,99%), nhưng khoảng cách này thu hẹp còn 0,34 điểm khi freeze một phần có LR riêng ($85{,}69 \pm 0{,}16\%$ so với $86{,}03 \pm 0{,}14\%$, năm seed mỗi bên, Welch $t = -3{,}47$, $p \approx 0{,}009$: nhỏ nhưng có ý nghĩa), với chỉ khoảng 44% thời gian GPU và 20% bộ nhớ huấn luyện; ở DeiT-S còn cao hơn 0,54 điểm (88,27% so với 87,73%). Linear probing rẻ nhất nhưng thấp hơn khoảng 17 điểm. Độ tin cậy cao (3 seed, các khác biệt lớn hơn độ lệch chuẩn nhiều lần), và việc freeze một phần vượt full fine-tuning ở DeiT-S bền với việc đổi LR; layer-wise LR decay chỉ thu hẹp một phần khoảng cách (bằng chứng yếu cho giả thuyết các block thấp bị cập nhật quá mức).

- **RQ2 – vai trò của pretraining.** Pretraining là yếu tố quan trọng nhất: +11,4 điểm với ResNet-50 và +53,2 điểm với DeiT-S ở 100% dữ liệu, và khoảng cách lớn hơn khi dữ liệu ít hơn (48 và 64 điểm ở 10% dữ liệu). Một phần của khoảng cách ở DeiT-S là do 40 epoch chưa đủ (34,72% lên 67,52% khi huấn luyện 100 epoch). Độ tin cậy trung bình (một seed cho from-scratch và ablation).

- **RQ3 – hiệu năng và chi phí.** Freeze một phần của DeiT-S là điểm đánh đổi tốt nhất trong thực nghiệm (độ chính xác cao nhất, 122,8 s/epoch, 1 336 MB). Ở suy luận, ResNet-50 và DeiT-S gần như tương đương; GMACs không dự đoán tốt độ trễ. Thời gian thực mỗi epoch bị giới hạn bởi bộ nạp dữ liệu nên không phản ánh lợi ích của freeze.

- **RQ4 – CNN và Transformer.** Có pretraining, DeiT-S vượt ResNet-50 trong giao thức,

nhưng khoảng cách phụ thuộc vào công thức: nó thu hẹp khi ResNet-50 có LR riêng cho freeze một phần (4,2 xuống 2,9 điểm) và còn 0,3 điểm khi cả hai bỏ Mixup/CutMix (một seed); cặp ConvNeXt-T/Swin-T lại ngang nhau ($\approx$ 88,9%); không có pretraining thì CNN vượt rõ. Hai họ mắc lỗi ở các cặp lớp rất giống nhau, và ensemble giữa họ chỉ hơn mô hình tốt nhất khoảng 1 điểm (hai họ sai ở các ảnh khác nhau một phần, Jaccard 0,48).

Phần mở rộng cho thấy các mô hình đều thiếu tự tin (ECE 13–24% trước, dưới 3% sau temperature scaling với mô hình full fine-tuning) và nguyên nhân chính là các mục tiêu mềm của Mixup/CutMix và label smoothing (bỏ cả hai đưa ECE về 2–3%); full fine-tuning kém bền với nhiễu Gauss ở cả hai kiến trúc và trên cả ba seed, và nhãn huấn luyện nhiễu của Food-101 giới hạn độ chính xác trên tập huấn luyện ở khoảng 89–91%.

9.2 Hạn chế và hướng phát triển

- **Chưa kiểm chứng trùng lặp với dữ liệu pretrain.** Nhóm không kiểm tra độc lập sự trùng lặp ở cấp ảnh giữa Food-101 và ImageNet-1k, nên lợi thế của pretrain có thể một phần phản ánh sự gần gũi về miền dữ liệu, chứ không hoàn toàn là khả năng chuyển giao sang dữ liệu chưa từng thấy.
- **Công thức pretrain khác nhau** giữa trọng số CNN và Transformer (Chương 3), nên khác biệt giữa hai họ không hoàn toàn quy được cho kiến trúc; kết quả của cặp ConvNeXt-T/Swin-T cho thấy kết luận phụ thuộc vào cặp được chọn.
- **From-scratch 40 epoch** chưa đủ để ViT hội tụ; đối chứng 100 epoch chỉ chạy cho DeiT-S và vẫn chưa bằng ResNet-50, nên khoảng cách thật ở lịch dài chưa được xác định.
- **Một seed** cho from-scratch, ablation và các mô hình mở rộng, nên các khác biệt nhỏ ở đó không được diễn giải; phương sai giữa seed chỉ đo ở cấu hình pretrained của hai mô hình chính.
- **Công thức augmentation cố định, không tinh chỉnh.** RandAugment nhẹ kết hợp Mixup/CutMix cho mọi batch có thể là điều chuẩn hóa quá mạnh cho lịch ngắn, và có thể là nguyên nhân khiến mô hình thiếu tự tin; nhóm không ablate từng thành phần.
- **Hầu hết thí nghiệm bổ sung chỉ có một seed** (ngoại lệ: cặp ResNet-50 freeze một phần ở LR riêng / full fine-tuning có năm seed mỗi bên) và được chạy sau khi đã thấy kết quả giao thức (Mục 5.6); quy tắc quyết định được đặt trước khi chạy, nhưng chúng không thay thế kết quả giao thức. Lượt quét ngắn còn thiên về LR lớn: ở ResNet-50 full fine-tuning và linear probing, LR cao hơn thắng ở 3 epoch nhưng không thắng ở 10 epoch.

- **Quét learning rate ngắn** (3 epoch, 20% dữ liệu) và ở 8 trong 12 nhóm (mô hình, chế độ), LR tốt nhất vẫn nằm ở biên lưới sau khi đã mở rộng thêm một mức, nên LR chọn được có thể chưa tối ưu; các lần chạy 10 epoch bổ sung cho thấy điều này không đổi kết quả với linear probing và full fine-tuning của ResNet-50 và DeiT-S freeze một phần, nhưng đổi kết quả của ResNet-50 freeze một phần (LR cao nhất đã thử vẫn là biên). Validation lấy từ phần train nhiễu nên thấp hơn test và chỉ dùng chọn siêu tham số.
- **Chỉ một tập dữ liệu và một loại GPU** (Tesla T4 trên Kaggle); độ chính xác không bảo đảm trùng từng bit giữa các lần chạy trên phần cứng khác. Chi phí được đo kiểm soát chỉ trên GPU vì thời gian thực bị giới hạn bởi bộ nạp dữ liệu.
- **Độ bền vững và diễn giải** dùng tập con 50 ảnh mỗi lớp và một seed; Grad-CAM là định tính và không phải bằng chứng nhân quả.

Hướng phát triển. Kiểm chứng mức trùng lặp ở cấp ảnh với ImageNet-1k; tinh chỉnh công thức augmentation (và tách ảnh hưởng của Mixup/label smoothing lên hiệu chỉnh); lịch huấn luyện dài hơn cho ViT; chạy nhiều seed cho các cấu hình from-scratch và ablation; mở rộng sang tập dữ liệu thứ hai.

TÀI LIỆU THAM KHẢO

[1] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In *European Conference on Computer Vision (ECCV)*, 2014. `https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/`.

[2] Ekin D. Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V. Le. RandAugment: Practical automated data augmentation with a reduced search space. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020. arXiv:1909.13719.

[3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.11929.

[4] ETH Zürich Computer Vision Lab. ethz/food101 (dataset card, licensing information). Hugging Face Datasets. Truy cập 21/10/2026.

[5] Foodspotting. Terms of service (bản lưu trữ 23/12/2017; hiệu lực từ 17/10/2012). Internet Archive.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017. arXiv:1706.04599.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[8] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1903.12261.

[9] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021. arXiv:2103.14030.

[10] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. arXiv:2201.03545.

[11] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

[12] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *IEEE International Conference on Computer Vision (ICCV)*, 2017. arXiv:1610.02391.

[13] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. arXiv:1512.00567.

[14] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning (ICML)*, 2021. arXiv:2012.12877.

[15] Ross Wightman. PyTorch image models (timm), 2019. `https://github.com/huggingface/pytorch-image-models`.

[16] Ross Wightman, Hugo Touvron, and Hervé Jégou. ResNet strikes back: An improved training procedure in timm. *arXiv preprint arXiv:2110.00476*, 2021.

[17] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. CutMix: Regularization strategy to train strong classifiers with localizable features. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. arXiv:1905.04899.

[18] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.09412.

Phụ lục A

Ví dụ tệp cấu hình một lần chạy

Mỗi lần chạy có một tệp YAML được commit; chỉ các giá trị ảnh hưởng đến kết quả khoa học nằm trong tệp này. Đường dẫn dữ liệu và checkpoint đi qua biến môi trường. Trong tệp YAML đã commit, `lr_peak` là `auto`: giá trị được lấy từ kết quả quét LR (Mục 5.4, tệp `lr_choice.json`) khi chạy và được ghi vào `results/<run_id>.json`; ví dụ dưới đây hiển thị giá trị đã giải ($9 \cdot 10^{-4}$ cho ResNet-50 full fine-tuning). Tệp đầy đủ còn có các khóa `kind`, `betas`, `eps`, `workers`, `img_size`, `llrd` và `eval`; ở đây chỉ trích các giá trị chính.

```
run_id: m_resnet50_full_s0
model: resnet50.a1_in1k
pretrained: true
mode: full                # scratch | head | partial | full
epochs: 10
seed: 0
data_fraction: 1.0        # 0.1 | 0.25 | 0.5 | 1.0 (ablation)
batch_size: 64
lr_peak: auto             # resolved from lr_choice.json at run time (here 0.0009), stored in results/<
warmup_frac: 0.05
min_lr_frac: 0.01
weight_decay: 0.05
drop_rate: 0.0
drop_path_rate: 0.0
amp: fp16
augmentation:
  random_resized_crop:
    size: 224
    scale: [0.25, 1.0]
    ratio: [0.75, 1.3333]
  hflip_p: 0.5
  randaugment:
    num_ops: 2
    magnitude: 9
    num_magnitude_bins: 31
```

```
  mixup:
    mixup_alpha: 0.8
    cutmix_alpha: 1.0
    prob: 1.0
    switch_prob: 0.5
    label_smoothing: 0.1
split_json_sha256_16: 75ed81bfe380c089
```

Phụ lục B

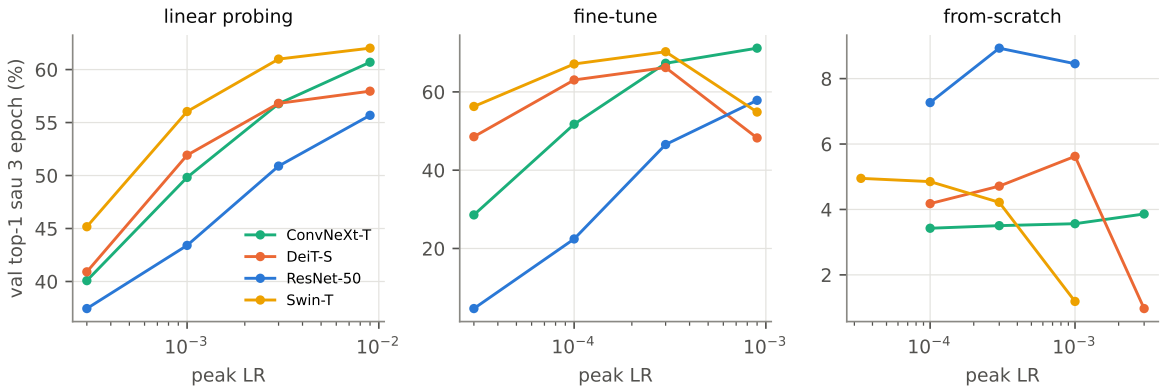
Kết quả thực nghiệm bổ sung

B.1 Quét learning rate

Top-1 validation (%) ở epoch cuối của lượt quét (3 epoch, 20% dữ liệu); dấu * đánh dấu mức LR được chọn. Các lưới có bốn mức là các lưới đã được mở rộng thêm một mức do LR tốt nhất nằm ở biên (Mục 5.4).

Bảng B.1: Kết quả quét learning rate.

Kiến trúc	Chế độ	Lưới LR	Top-1 validation (%) theo từng mức LR			
ConvNeXt-T	From-scratch	10^{-4}, $3\cdot10^{-4}$, 10^{-3}, $3\cdot10^{-3}$	3,43	3,50	3,56	3,86*
ConvNeXt-T	Linear probing	$3\cdot10^{-4}$, 10^{-3}, $3\cdot10^{-3}$, $9\cdot10^{-3}$	40,08	49,82	56,77	60,69*
ConvNeXt-T	Full fine-tuning	$3\cdot10^{-5}$, 10^{-4}, $3\cdot10^{-4}$, $9\cdot10^{-4}$	28,57	51,74	67,31	71,17*
DeiT-S	From-scratch	10^{-4}, $3\cdot10^{-4}$, 10^{-3}, $3\cdot10^{-3}$	4,18	4,71	5,62*	0,97
DeiT-S	Linear probing	$3\cdot10^{-4}$, 10^{-3}, $3\cdot10^{-3}$, $9\cdot10^{-3}$	40,91	51,92	56,81	57,96*
DeiT-S	Full fine-tuning	$3\cdot10^{-5}$, 10^{-4}, $3\cdot10^{-4}$, $9\cdot10^{-4}$	48,55	63,05	66,22*	48,24
ResNet-50	From-scratch	10^{-4}, $3\cdot10^{-4}$, 10^{-3}	7,27	8,93*	8,46	
ResNet-50	Linear probing	$3\cdot10^{-4}$, 10^{-3}, $3\cdot10^{-3}$, $9\cdot10^{-3}$	37,45	43,41	50,89	55,68*
ResNet-50	Full fine-tuning	$3\cdot10^{-5}$, 10^{-4}, $3\cdot10^{-4}$, $9\cdot10^{-4}$	4,65	22,44	46,55	57,84*
Swin-T	From-scratch	$3.33\cdot10^{-5}$, 10^{-4}, $3\cdot10^{-4}$, 10^{-3}	4,95*	4,85	4,22	1,19
Swin-T	Linear probing	$3\cdot10^{-4}$, 10^{-3}, $3\cdot10^{-3}$, $9\cdot10^{-3}$	45,17	56,04	60,99	62,02*
Swin-T	Full fine-tuning	$3\cdot10^{-5}$, 10^{-4}, $3\cdot10^{-4}$, $9\cdot10^{-4}$	56,28	67,13	70,26*	54,85



Hình B.1: Top-1 validation theo learning rate.

B.1.1 Điểm quét bổ sung sau giao thức

Quét LR ngắn (3 epoch, 20% dữ liệu) cho các nhóm ở biên lưới và cho freeze một phần, kèm các điểm tương ứng của giao thức (Mục 5.6). Dấu * là LR đã chọn trong giao thức; với freeze một phần giao thức dùng LR của full fine-tuning.

Bảng B.2: Điểm quét LR bổ sung: top-1 validation (%).

Kiến trúc	Chế độ	LR	Top-1 val (%)	Nguồn
DeiT-S	Linear probing	$3 \cdot 10^{-4}$	40,91	giao thức
DeiT-S	Linear probing	10^{-3}	51,92	giao thức
DeiT-S	Linear probing	$3 \cdot 10^{-3}$	56,81	giao thức
DeiT-S	Linear probing	$9 \cdot 10^{-3}$	57,96* (tốt nhất)	giao thức
DeiT-S	Linear probing	$2.7 \cdot 10^{-2}$	56,34	bổ sung
DeiT-S	Freeze một phần	10^{-4}	61,74	bổ sung
DeiT-S	Freeze một phần	$3 \cdot 10^{-4}$	69,47	bổ sung
DeiT-S	Freeze một phần	10^{-3}	71,37 (tốt nhất)	bổ sung
DeiT-S	Freeze một phần	$3 \cdot 10^{-3}$	63,66	bổ sung
ResNet-50	Linear probing	$3 \cdot 10^{-4}$	37,45	giao thức
ResNet-50	Linear probing	10^{-3}	43,41	giao thức
ResNet-50	Linear probing	$3 \cdot 10^{-3}$	50,89	giao thức
ResNet-50	Linear probing	$9 \cdot 10^{-3}$	55,68*	giao thức
ResNet-50	Linear probing	$2.7 \cdot 10^{-2}$	56,69 (tốt nhất)	bổ sung
ResNet-50	Freeze một phần	$3 \cdot 10^{-4}$	45,96	bổ sung
ResNet-50	Freeze một phần	$9 \cdot 10^{-4}$	58,26	bổ sung
ResNet-50	Freeze một phần	$2.7 \cdot 10^{-3}$	65,64 (tốt nhất)	bổ sung
ResNet-50	Full fine-tuning	$3 \cdot 10^{-5}$	4,65	giao thức
ResNet-50	Full fine-tuning	10^{-4}	22,44	giao thức
ResNet-50	Full fine-tuning	$3 \cdot 10^{-4}$	46,55	giao thức
ResNet-50	Full fine-tuning	$9 \cdot 10^{-4}$	57,84*	giao thức
ResNet-50	Full fine-tuning	$2.7 \cdot 10^{-3}$	60,14 (tốt nhất)	bổ sung

B.2 Kết quả từng seed

Bảng B.3: Top-1 test (%) của từng seed (cấu hình pretrained, 10 epoch) và độ lệch chuẩn.

Kiến trúc	Chế độ	Seed 0	Seed 1	Seed 2	Trung bình	Std
DeiT-S	Linear probing	70,21	70,16	69,99	70,12	0,11
DeiT-S	Freeze một phần	88,36	88,15	88,29	88,27	0,11
DeiT-S	Full fine-tuning	87,88	87,64	87,65	87,73	0,13
ResNet-50	Linear probing	69,26	69,48	69,01	69,25	0,24
ResNet-50	Freeze một phần	84,16	84,11	84,00	84,09	0,08
ResNet-50	Full fine-tuning	86,01	86,05	85,91	85,99	0,07

B.3 Chi phí và hiệu năng của chế độ freeze

Bảng B.4: Độ chính xác so với full fine-tuning và chi phí của từng chế độ.

Kiến trúc	Chế độ	Top-1 (%)	Δ so với full	Trainable	phút/epoch (thực tế)	Peak mem (MB, lần chạy thật)
ResNet-50	Linear probing	69,25	−16,74	0,9%	4,5	470
ResNet-50	Freeze một phần	84,09	−1,90	64,0%	4,9	646
ResNet-50	Full fine-tuning	85,99	+0,00	100,0%	5,2	3 161
DeiT-S	Linear probing	70,12	−17,61	0,2%	4,9	287
DeiT-S	Freeze một phần	88,27	+0,54	49,2%	4,7	1 336
DeiT-S	Full fine-tuning	87,73	+0,00	100,0%	4,8	2 502

Độ bền vững theo từng mức nhiễu, biểu đồ reliability và các ví dụ Grad-CAM và ảnh loss cao được trình bày ở Mục 6.6 (Hình 6.6, 6.5, 6.7, 6.8).

Phụ lục C

Khai báo sử dụng công cụ AI

Phụ lục này nêu công cụ AI đã được dùng trong bài tập lớn A1, dùng vào việc gì, và nhóm đã kiểm tra kết quả như thế nào. Nhóm chịu trách nhiệm về toàn bộ nội dung nộp.

C.1 Công cụ và phạm vi

Nhóm dùng **Claude** làm trợ lý lập trình và viết tài liệu. Các công việc cụ thể:

- **Mã nguồn** trong `A1/a1lib` và `A1/tests`: pipeline dữ liệu và cache, vòng huấn luyện, các chế độ freeze, lập kế hoạch lần chạy, worker chạy được tiếp tục sau khi bị ngắt, kho kết quả trên Hugging Face, đo chi phí (`cost_bench.py`), phân tích hậu kiểm (hiệu chỉnh, ensemble, độ bền vững, Grad-CAM) và các script sinh bảng, hình.
- **Thiết kế thực nghiệm**: bản nháp quy trình (`TEAM_PROTOCOL.md`), kế hoạch 77 lần chạy chính, quy tắc quét learning rate và các lần chạy bổ sung.
- **Báo cáo và trang A1**: soạn thảo và chỉnh sửa văn bản tiếng Việt, và đối chiếu từng con số trong báo cáo với tệp kết quả (`results/*.json`) và với đề bài. Qua các lần đối chiếu này, công cụ AI đã phát hiện và nhóm đã sửa các sai lệch như số lần chạy và số giờ GPU, số liệu thống kê tính từ số đã làm tròn, và số GMACs của DeiT-S bị thiếu phần attention hợp nhất.

C.2 Những gì AI không làm

- **Không tạo số liệu.** Mọi số trong báo cáo đến từ các tệp kết quả do mã nguồn sinh ra khi nhóm chạy thí nghiệm; AI không chạy các thí nghiệm huấn luyện. Các lần chạy được nhóm thực hiện trên Kaggle (Tesla T4) và lưu trong kho Hugging Face của nhóm.

- **Không dùng dữ liệu hay hình ảnh do AI sinh.** Ảnh trong báo cáo là ảnh của Food-101; các hình biểu đồ do mã nguồn của nhóm vẽ từ kết quả.
- **Không thay thế trọng số hay mô hình.** Trọng số pretrained lấy từ `timm`, như nêu ở Chương 3.

C.3 Cách nhóm kiểm tra

- Chạy bộ kiểm thử đơn vị (`python -m unittest discover`) và một lần chạy thử (smoke test) trước khi chạy thật.
- Chạy lại bước phân tích từ các tệp kết quả đã commit, rồi so sánh bảng và hình do mã sinh ra với văn bản báo cáo.
- Xem lại các hình kết quả (Grad-CAM, ảnh loss cao nhất, đường reliability) và kiểm tra chúng khớp với mô tả.
- Các sai lệch do AI gây ra mà nhóm phát hiện (như số làm tròn hai lần) đã được sửa trong bản nộp.

C.4 Hạn chế

Văn bản do AI soạn có thể sai ở chi tiết nên nhóm chỉ giữ lại phần đã được đối chiếu với kết quả hoặc nguồn.